

Mathematical Foundations, Architecture Diagrams, and State Analysis of Modern LLM Training Methods

Chief Strategist J

June 13, 2026

Contents

I	Foundation Training Methods	2
1	Pretraining	2
1.1	Mathematical Foundation	2
1.2	Architecture Flow Diagram	2
1.3	State Transition Table	2
2	Continued Pretraining (CPT)	3
2.1	Mathematical Foundation	3
2.2	Architecture Flow Diagram	3
2.3	State Transition Table	3
3	Domain Adaptive Pretraining (DAPT)	4
3.1	Mathematical Foundation	4
3.2	Architecture Flow Diagram	4
3.3	State Transition Table	4
4	Task Adaptive Pretraining (TAPT)	5
4.1	Mathematical Foundation	5
4.2	Architecture Flow Diagram	5
4.3	State Transition Table	5
II	Supervised Fine-Tuning Methods	6
5	Supervised Fine-Tuning (SFT)	6
5.1	Mathematical Foundation	6
5.2	Architecture Flow Diagram	6
5.3	State Transition Table	6
6	Instruction Tuning	7
6.1	Mathematical Foundation	7
6.2	Architecture Flow Diagram	7
6.3	State Transition Table	7
7	Multi-Task Training	8
7.1	Mathematical Foundation	8
7.2	Architecture Flow Diagram	8
7.3	State Transition Table	8
8	Chain-of-Thought SFT	9
8.1	Mathematical Foundation	9
8.2	Architecture Flow Diagram	9
8.3	State Transition Table	9

9	Step-by-Step SFT	10
9.1	Mathematical Foundation	10
9.2	Architecture Flow Diagram	10
9.3	State Transition Table	10
10	Process Supervision	11
10.1	Mathematical Foundation	11
10.2	Architecture Flow Diagram	11
10.3	State Transition Table	11
III	Parameter-Efficient Fine-Tuning (PEFT)	12
11	Low-Rank Adaptation (LoRA)	12
11.1	Mathematical Foundation	12
11.2	Architecture Flow Diagram	12
11.3	State Transition Table	12
11.4	Memory Complexity Analysis	13
11.5	Detailed Operational Analysis of LoRA Variables	13
11.6	The Physical and Philosophical Foundations of LoRA	20
11.6.1	Philosophical Foundations: Occam’s Razor and the Manifold Hypothesis	20
11.6.2	Internal Mechanics and Representation Space Geometry	21
11.6.3	Visualizing the Calculations	21
11.7	The Physical and Philosophical Foundations of LoRA	21
11.7.1	Philosophical Foundations: Occam’s Razor and the Manifold Hypothesis	21
11.7.2	Internal Mechanics and Representation Space Geometry	22
11.7.3	Visualizing the Calculations	23
11.8	The Physical and Philosophical Foundations of LoRA	23
11.8.1	Philosophical Foundations: Occam’s Razor and the Manifold Hypothesis	23
11.8.2	Internal Mechanics and Representation Space Geometry	24
11.8.3	Visualizing the Calculations	24
11.9	The Physical and Philosophical Foundations of LoRA	25
11.9.1	Philosophical Foundations: Occam’s Razor and the Manifold Hypothesis	25
11.9.2	Internal Mechanics and Representation Space Geometry	26
11.9.3	Visualizing the Calculations	26
11.10	The Physical and Philosophical Foundations of LoRA	27
11.10.1	Philosophical Foundations: Occam’s Razor and the Manifold Hypothesis	27
11.10.2	Internal Mechanics and Representation Space Geometry	28
11.10.3	Visualizing the Calculations	28
11.11	The Physical and Philosophical Foundations of LoRA	28
11.11.1	Philosophical Foundations: Occam’s Razor and the Manifold Hypothesis	28
11.11.2	Internal Mechanics and Representation Space Geometry	29
11.11.3	Visualizing the Calculations	30
11.12	The Physical and Philosophical Foundations of LoRA	30
11.12.1	Philosophical Foundations: Occam’s Razor and the Manifold Hypothesis	30
11.12.2	Internal Mechanics and Representation Space Geometry	31
11.12.3	Visualizing the Calculations	31
11.13	LoRA CLRS-Style Algorithms	32
11.13.1	Analysis of LORA-FORWARD	32
11.13.2	Analysis of LORA-BACKWARD	32
11.13.3	Analysis of LORA-MERGE	33
11.13.4	Analysis of LORA-ADAMW-UPDATE	33
12	Quantized LoRA (QLoRA)	34
12.1	Mathematical Foundation	34
12.2	Architecture Flow Diagram	34
12.3	State Transition Table	34
13	Adaptive LoRA (AdaLoRA)	35
13.1	Mathematical Foundation	35
13.2	Architecture Flow Diagram	35
13.3	State Transition Table	35

14 Weight-Decomposed LoRA (DoRA)	36
14.1 Mathematical Foundation	36
14.2 Architecture Flow Diagram	36
14.3 State Transition Table	36
15 IA³ (Infused Adapter by Inhibiting and Amplifying Inner Activations)	37
15.1 Mathematical Foundation	37
15.2 Architecture Flow Diagram	37
15.3 State Transition Table	37
16 Adapter Tuning	38
16.1 Mathematical Foundation	38
16.2 Architecture Flow Diagram	38
16.3 State Transition Table	38
17 Prefix Tuning	39
17.1 Mathematical Foundation	39
17.2 Architecture Flow Diagram	39
17.3 State Transition Table	39
18 Prompt Tuning	40
18.1 Mathematical Foundation	40
18.2 Architecture Flow Diagram	40
18.3 State Transition Table	40
19 P-Tuning v2	41
19.1 Mathematical Foundation	41
19.2 Architecture Flow Diagram	41
19.3 State Transition Table	41

Part I

Foundation Training Methods

Pretraining

Mathematical Foundation

Causal language modeling models the probability of a sequence $x = (x_1, \dots, x_T)$ as the product of autoregressive conditional probabilities:

$$P(x) = \prod_{t=1}^T P(x_t \mid x_{<t}) \quad (1)$$

The hidden representation $h_t^{(l)}$ at layer l and token position t is computed using multi-head causal self-attention. The causal attention mask matrix $M \in \mathbb{R}^{T \times T}$ is defined as:

$$M_{i,j} = \begin{cases} 0 & \text{if } j \leq i \\ -\infty & \text{if } j > i \end{cases} \quad (2)$$

This mask is added directly to the scaled query-key dot products:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^\top}{\sqrt{d_k}} + M \right) V \quad (3)$$

where $Q = XW_Q$, $K = XW_K$, $V = XW_V$. This prevents information leakage from future tokens. At the output layer, the logits $z_t \in \mathbb{R}^{|V|}$ are computed via $z_t = h_t^{(L)} W_{\text{vocab}}$. To maintain numerical stability, softmax is computed by subtracting the maximum logit:

$$P(x_t = v \mid x_{<t}) = \frac{\exp(z_{t,v} - \max_k z_{t,k})}{\sum_{j \in V} \exp(z_{t,j} - \max_k z_{t,k})} \quad (4)$$

The causal model parameters θ are trained by minimizing the cross-entropy loss over a dataset $\mathcal{D}$:

$$\mathcal{L}_{\text{PT}}(\theta) = -\frac{1}{|\mathcal{D}|} \sum_{x \in \mathcal{D}} \sum_{t=1}^T \log P_\theta(x_t \mid x_{<t}) \quad (5)$$

Updates are made using the AdamW optimizer. Let $g_t = \nabla_\theta \mathcal{L}_t$ be the gradient at step t . The update equations are:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \quad (6)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \quad (7)$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t} \quad (8)$$

$$\theta_t = \theta_{t-1} - \eta \left(\frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} + \lambda \theta_{t-1} \right) \quad (9)$$

where η is the learning rate, λ is the weight decay rate, β_1, β_2 are the exponential decay rates, and ϵ prevents division by zero. A cosine learning rate schedule with linear warmup is employed:

$$\eta_t = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min}) \left(1 + \cos \left(\frac{t - T_{\text{warmup}}}{T_{\max} - T_{\text{warmup}}} \pi \right) \right) \quad (10)$$

Architecture Flow Diagram

State Transition Table

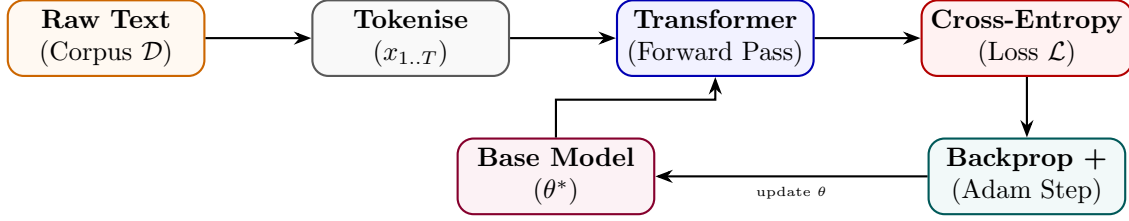


Figure 1: Pretraining pipeline: corpus sampling, forward pass, loss computation, and weight update loop.

Property	Before	After
Parameter source	Random init $\theta_0 \sim \mathcal{N}(0, \sigma^2)$	Converged weights θ^*
Training corpus	Not consumed	Trillions of tokens processed
Perplexity	Very high ($> 10^3$)	Low (< 20 on held-out data)
World knowledge	None	Broad language & factual knowledge
requires_grad	True (all params)	True (all params)

Table 1: State properties before and after pretraining.

Continued Pretraining (CPT)

Mathematical Foundation

Continued Pretraining (CPT) initializes parameters with a pretrained checkpoint θ_{base} and continues autoregressive training. To mitigate catastrophic forgetting, training batches are sampled from a mixture of the new domain-specific corpus $\mathcal{D}_{\text{new}}$ and a replay buffer of general pretraining data $\mathcal{D}_{\text{replay}}$:

$$x \sim \lambda \mathcal{D}_{\text{new}} + (1 - \lambda) \mathcal{D}_{\text{replay}} \quad (11)$$

where $\lambda \in [0, 1]$ is the mixing ratio. The CPT objective minimizes the cross-entropy loss over this mixed distribution:

$$\mathcal{L}_{\text{CPT}}(\theta) = -\mathbb{E}_{x \sim \mathcal{D}_{\text{mix}}} \sum_{t=1}^T \log P_{\theta}(x_t | x_{<t}) \quad (12)$$

The parameter updates follow the AdamW optimizer starting from $\theta_0 = \theta_{\text{base}}$, typically using a warm-restart cosine learning rate schedule:

$$\eta_t = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min}) \left(1 + \cos \left(\frac{t}{T_{\max}} \pi \right) \right) \quad (13)$$

where the peak learning rate $\eta_{\max}$ is set to a lower value than during the initial pretraining phase to preserve general capabilities.

Architecture Flow Diagram

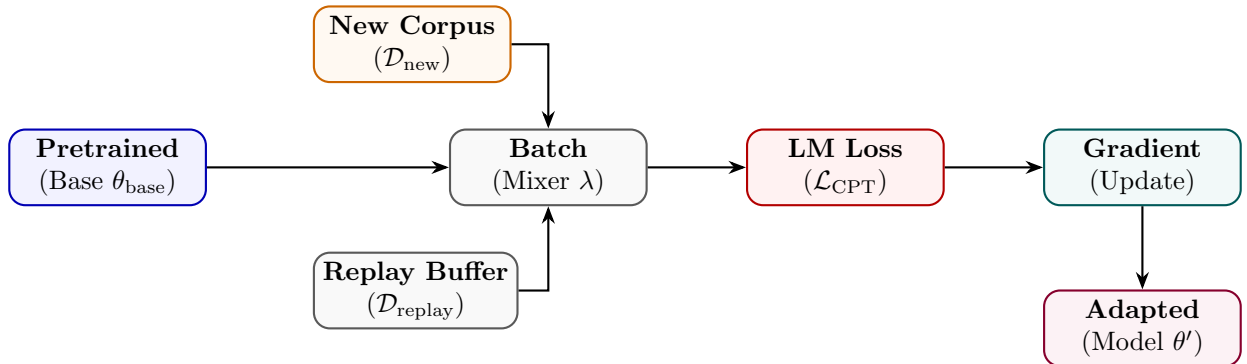


Figure 2: CPT mixes new corpus and replay buffer batches to update a pretrained checkpoint.

State Transition Table

Property	Before	After
Starting checkpoint	Pretrained θ_{base}	$\theta' = \theta_{\text{base}} + \Delta\theta$
New domain knowledge	Absent	Incorporated
Original knowledge	Retained	Partially retained via replay
Learning rate schedule	Finished	Warm-restart applied
Perplexity on new domain	High	Significantly reduced

Table 2: State properties before and after continued pretraining.

Domain Adaptive Pretraining (DAPT)

Mathematical Foundation

Domain Adaptive Pretraining (DAPT) adapts a general-domain pretrained model θ_{base} to a target domain (e.g., medical, legal, or code) by training on a large, unlabelled domain-specific corpus $\mathcal{D}_{\text{domain}}$. The objective function is the causal language modeling loss over the domain dataset:

$$\mathcal{L}_{\text{DAPT}}(\theta) = -\mathbb{E}_{x \sim \mathcal{D}_{\text{domain}}} \sum_{t=1}^T \log P_{\theta}(x_t | x_{<t}) \quad (14)$$

where parameters are initialized at $\theta_0 = \theta_{\text{base}}$. DAPT uses a lower learning rate than initial pretraining to prevent catastrophic disruption of base semantic features while allowing the model to adapt vocabulary representations and learn domain-specific concepts.

Architecture Flow Diagram

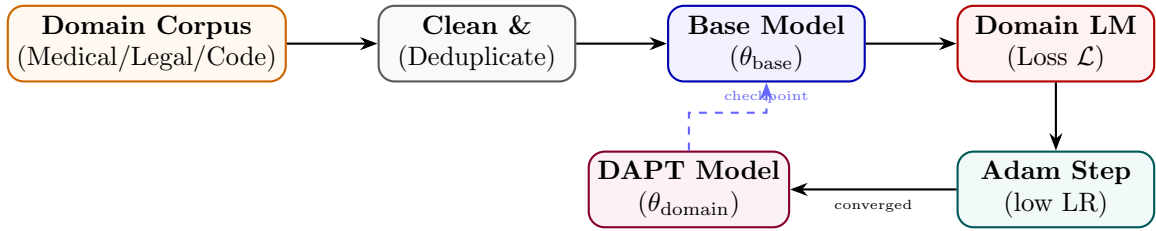


Figure 3: DAPT pipeline: domain corpus is cleaned, fed into the base model, and trained with standard LM loss.

State Transition Table

Property	Before	After
Domain vocabulary familiarity	Low	High
Domain perplexity	High ($\rho \gg 1$)	Low (near 1)
General language ability	Strong	Marginally reduced
Downstream task (same domain)	Weak	Significantly stronger
Training data type	General web text	Domain-specific unlabelled

Table 3: State properties before and after domain adaptive pretraining.

Task Adaptive Pretraining (TAPT)

Mathematical Foundation

Task Adaptive Pretraining (TAPT) optimizes a pretrained or domain-adapted checkpoint θ_{base} on a task-specific unlabelled dataset $\mathcal{D}_{\text{task}}$:

$$\mathcal{L}_{\text{TAPT}}(\theta) = -\mathbb{E}_{x \sim \mathcal{D}_{\text{task}}} \sum_{t=1}^T \log P_{\theta}(x_t \mid x_{<t}) \quad (15)$$

Since the task dataset size $|\mathcal{D}_{\text{task}}|$ is typically much smaller than a general or domain-specific corpus, TAPT uses very few epochs, a small learning rate, and regularizers (such as dropout) to prevent overfitting before final supervised fine-tuning.

Architecture Flow Diagram

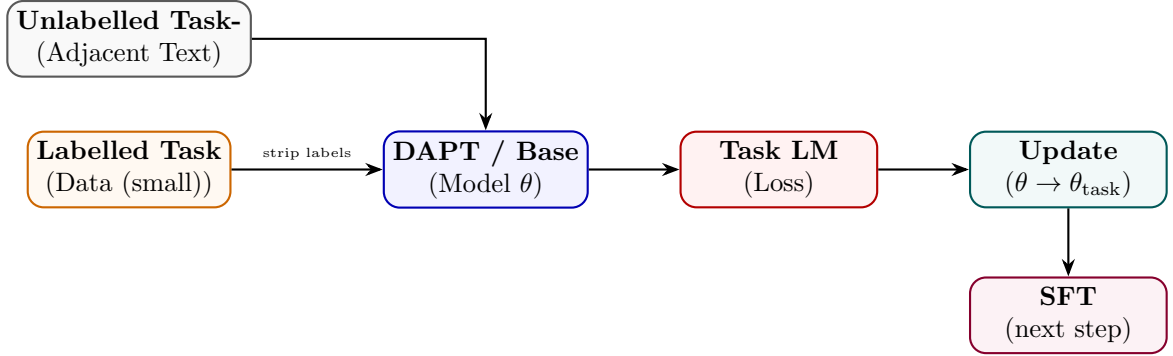


Figure 4: TAPT uses task-adjacent unlabelled text to bridge base model and supervised fine-tuning.

State Transition Table

Property	Before	After
Task vocabulary alignment	Partial	Strong
Corpus size	Millions of docs	Thousands of docs
Training epochs	1–3	10–100 (small corpus)
Downstream label efficiency	Requires more labels	Requires fewer labels
Transfer to other tasks	Good	Reduced (narrow focus)

Table 4: State properties before and after task adaptive pretraining.

Part II

Supervised Fine-Tuning Methods

Supervised Fine-Tuning (SFT)

Mathematical Foundation

SFT trains the model on structured prompt-response pairs $(x, y) \sim \mathcal{D}_{\text{SFT}}$ using teacher forcing. Let $s = [x; y]$ be the concatenated token sequence of length $M + U$, where $x = (s_1, \dots, s_M)$ is the prompt and $y = (s_{M+1}, \dots, s_{M+U})$ is the response. The key math lies in applying a target label mask $L \in \{-100, \mathcal{V}\}^{M+U}$ defined as:

$$L_t = \begin{cases} -100 & \text{if } t \leq M \\ s_t & \text{if } t > M \end{cases} \quad (16)$$

The cross-entropy loss function ignores targets set to -100 , directing gradient updates exclusively to the response generation tokens:

$$\mathcal{L}_{\text{SFT}}(\theta) = -\frac{1}{U} \sum_{t=M+1}^{M+U} \log P_{\theta}(s_t \mid s_{<t}) \quad (17)$$

This loss ignores the prompt distribution $P(x)$, preventing the model from degrading its output generation capability to match arbitrary prompts. The gradients only backpropagate through response tokens:

$$\nabla_{\theta} \mathcal{L}_{\text{SFT}} = -\frac{1}{U} \sum_{t=M+1}^{M+U} \nabla_{\theta} \log P_{\theta}(s_t \mid s_{<t}) \quad (18)$$

Architecture Flow Diagram

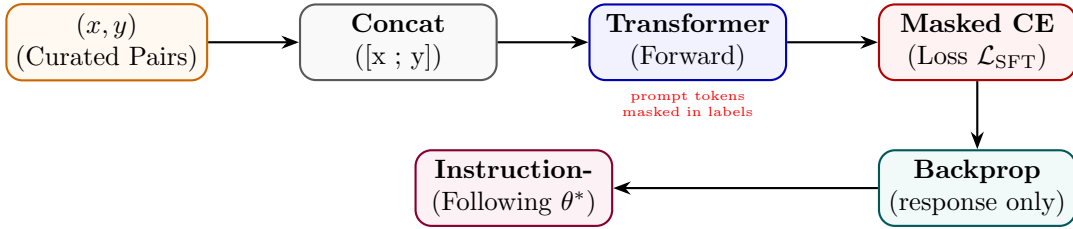


Figure 5: SFT pipeline with prompt-masked loss: gradients flow only through response tokens.

State Transition Table

Property	Before	After
Behaviour	Completion only	Instruction following
Loss region	N/A	Response tokens only
Label masking	None	Prompt masked to -100
Data format	Raw text	(instruction, response) pairs
Output quality	Random completion	Task-aligned responses

Table 5: State properties before and after supervised fine-tuning.

Instruction Tuning

Mathematical Foundation

Instruction Tuning exposes the model to multi-turn conversational patterns. A prompt template $T(I, x)$ converts an instruction I and context x into structured sequences. Turn boundaries are defined using chat-formatting systems (like ChatML). A single sequence layout is:

$$S = \langle \text{user} \rangle I \parallel x \langle \text{assistant} \rangle y \langle \text{eos} \rangle \quad (19)$$

Let $\mathcal{D}_{\text{inst}}$ consist of instructions spanning hundreds of tasks. The model parameters θ are optimized on the joint multi-task dataset:

$$\mathcal{L}_{\text{IT}}(\theta) = -\mathbb{E}_{(I, x, y) \sim \mathcal{D}_{\text{inst}}} \sum_{t=1}^{|y|} \log P_{\theta}(y_t \mid T(I, x), y_{<t}) \quad (20)$$

We mask all prompt-side tokens, including conversational format tags (like $\langle \text{user} \rangle$ and instruction text), ensuring parameters are trained only on synthesizing assistant responses:

$$\mathcal{L}_{\text{IT}}(\theta) = -\frac{1}{|y|} \sum_{i=1}^{|T(I, x)| + |y|} M_i \log P_{\theta}(S_i \mid S_{<i}), \quad M_i = \mathbb{I}(i > |T(I, x)|) \quad (21)$$

Architecture Flow Diagram

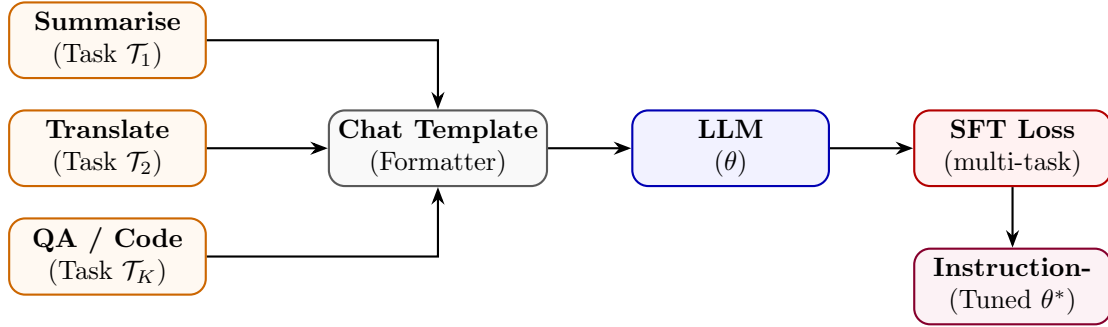


Figure 6: Instruction tuning over diverse task types funnelled through a unified chat template.

State Transition Table

Property	Before	After
Zero-shot generalisation	Poor	Strong across task types
Task diversity in training	N/A	K heterogeneous tasks
Input format	Free-form text	Structured chat template
Prompt sensitivity	High	Reduced
Held-out task performance	Weak	Strong (FLAN, InstructGPT)

Table 6: State properties before and after instruction tuning.

Multi-Task Training

Mathematical Foundation

Multi-Task Training aggregates losses from K distinct tasks. A simple summation can result in tasks with large gradient scales dominating the parameter updates. To resolve this, we use the GradNorm algorithm. Let W be the shared parameter weights (e.g., the last transformer block weights). We define the gradient norm for task k at training step t as:

$$G_k(t) = \|\nabla_W(w_k(t)\mathcal{L}_k(t))\|_2 \quad (22)$$

Let $\bar{G}(t) = \frac{1}{K} \sum_k G_k(t)$ be the average gradient norm. The target gradient norm for task k balances task difficulties:

$$\hat{G}_k(t) = \bar{G}(t) \times \left(\frac{\tilde{\mathcal{L}}_k(t)}{\frac{1}{K} \sum_j \tilde{\mathcal{L}}_j(t)} \right)^\alpha \quad (23)$$

where $\tilde{\mathcal{L}}_k(t) = \mathcal{L}_k(t)/\mathcal{L}_k(0)$ is the task loss ratio indicating task training progress, and α is a hyperparameter managing structural gradient balancing. The task weights $w_k(t)$ are updated by minimizing the L1 distance between $G_k(t)$ and $\hat{G}_k(t)$:

$$\mathcal{L}_{\text{grad}}(w_1, \dots, w_K; t) = \sum_{k=1}^K |G_k(t) - \hat{G}_k(t)| \quad (24)$$

The final joint parameter objective is:

$$\mathcal{L}_{\text{MT}}(\theta) = \sum_{k=1}^K w_k \mathcal{L}_k(\theta) \quad (25)$$

Architecture Flow Diagram

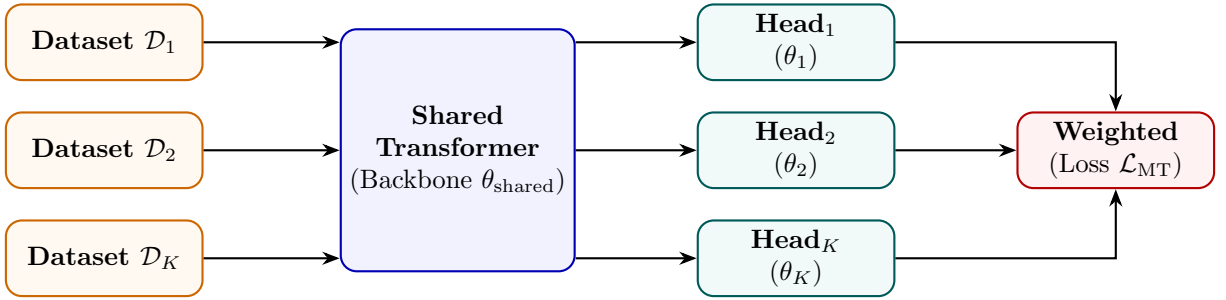


Figure 7: Multi-task training: each dataset feeds task-specific heads on a shared backbone.

State Transition Table

Property	Before	After
Backbone specialisation	Single task	K tasks jointly
Task heads	1	K heads
Generalisation	Narrow	Broad
Data efficiency	Per-task only	Cross-task transfer
Gradient conflicts	N/A	Managed via λ_k

Table 7: State properties before and after multi-task training.

Chain-of-Thought SFT

Mathematical Foundation

CoT SFT trains models to output intermediate reasoning steps before generating the final answer. Let x be the prompt, $c = (c_1, \dots, c_M)$ be the reasoning chain tokens, and $y = (y_1, \dots, y_U)$ be the final answer tokens. The joint probability factorization is:

$$P(c, y | x) = \prod_{i=1}^M P_{\theta}(c_i | x, c_{<i}) \prod_{j=1}^U P_{\theta}(y_j | x, c, y_{<j}) \quad (26)$$

The model is trained by minimizing the joint loss:

$$\mathcal{L}_{\text{CoT}}(\theta) = -\beta \sum_{i=1}^M \log P_{\theta}(c_i | x, c_{<i}) - (1 - \beta) \sum_{j=1}^U \log P_{\theta}(y_j | x, c, y_{<j}) \quad (27)$$

During autoregressive generation, we sample tokens using Temperature-scaled softmax and Nucleus (top- p) filtering. The logits z_t are modified as:

$$\tilde{z}_{t,i} = \frac{z_{t,i}}{T} \quad (28)$$

where $T > 0$ is the temperature. The top- p vocabulary subset $\mathcal{V}^{(p)}$ at step t is the smallest set satisfying:

$$\sum_{v \in \mathcal{V}^{(p)}} P(v | x, s_{<t}) \geq p \quad (29)$$

Tokens outside $\mathcal{V}^{(p)}$ are assigned probability 0 before final selection.

Architecture Flow Diagram

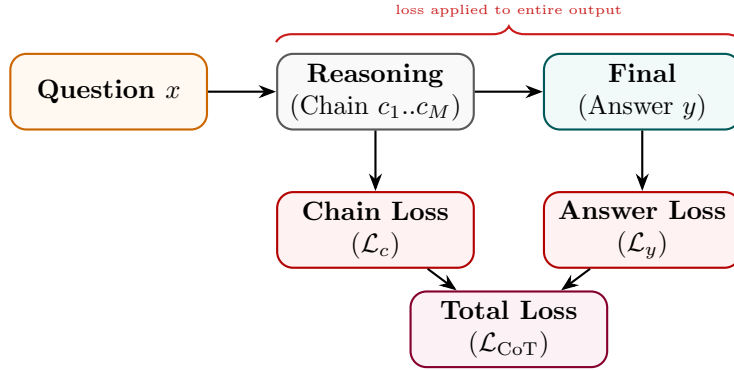


Figure 8: CoT SFT: loss is computed over both the reasoning chain and the final answer.

State Transition Table

Property	Before	After
Output format	Direct answer	Reasoning chain + answer
Training signal per example	$ y $ tokens	$ c + y $ tokens
Multi-step reasoning	Weak	Strong
Data annotation cost	Low	High (chain annotation)
Accuracy on complex tasks	Low	Substantially improved

Table 8: State properties before and after Chain-of-Thought SFT.

Step-by-Step SFT

Mathematical Foundation

Step-by-Step SFT formats the reasoning chain into discrete steps separated by a boundary token $T_{\text{step}} = [\text{STEP}]$. Let the response contain S steps: $s = (s_1, T_{\text{step}}, s_2, T_{\text{step}}, \dots, s_S, T_{\text{step}})$. The step-level loss is:

$$\mathcal{L}_{\text{SBS}}(\theta) = - \sum_{k=1}^S \sum_{t=1}^{|s_k|} \log P_{\theta}(s_{k,t} \mid x, s_{<k}, s_{k,<t}) - \sum_{k=1}^S \log P_{\theta}(T_{\text{step}} \mid x, s_{\leq k}) \quad (30)$$

We define the average step perplexity PPL_k as:

$$\text{PPL}_k = \exp \left(- \frac{1}{|s_k|} \sum_{t=1}^{|s_k|} \log P_{\theta}(s_{k,t} \mid x, s_{<k}, s_{k,<t}) \right) \quad (31)$$

Steps with high perplexity ($\text{PPL}_k > \tau$, where τ is a threshold) indicate low model confidence and can be flagged for step-level revision during generation. The model is trained to minimize the combined cross-entropy loss over all steps.

Architecture Flow Diagram

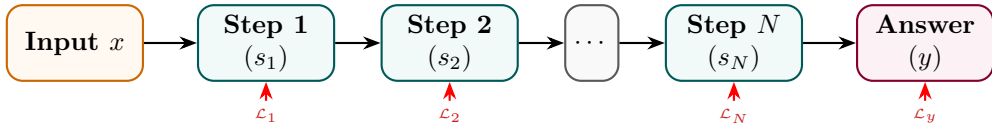


Figure 9: Step-by-Step SFT: each numbered step and the final answer contribute to the total loss.

State Transition Table

Property	Before	After
Solution structure	Unstructured	Numbered procedural steps
Step delimiter tokens	Not in vocabulary	Learned as first-class tokens
Interpretability	Low	Step-level traceability
Annotation format	Raw answer	$(x, [s_1, \dots, s_N], y)$
Task decomposition ability	Implicit	Explicit

Table 9: State properties before and after Step-by-Step SFT.

Process Supervision

Mathematical Foundation

Process Supervision trains a Process Reward Model (PRM) using step-level binary correctness labels. Let x be the problem description, and $s_1, \dots, s_S$ be the generated steps. Human or model-based annotations assign correctness labels $r_k \in \{0, 1\}$ for each step s_k . The PRM model outputs correctness probabilities by applying a logistic sigmoid function to the step boundary token representation:

$$\hat{r}_k = \sigma(\text{PRM}_\phi(h_{t_k})) = \frac{1}{1 + \exp(-\text{PRM}_\phi(h_{t_k}))} \quad (32)$$

where h_{t_k} is the transformer hidden output at index t_k representing the step boundary token. The PRM parameters ϕ are optimized using binary cross entropy:

$$\mathcal{L}_{\text{PRM}}(\phi) = - \sum_{k=1}^S [r_k \log \hat{r}_k + (1 - r_k) \log(1 - \hat{r}_k)] \quad (33)$$

During inference, candidate sequences are generated. The total sequence correctness score is calculated as the product of step correctness probabilities:

$$\text{Score}(s_{\leq S}) = \prod_{k=1}^S \hat{r}_k \quad (34)$$

This score is used to rank candidate rollouts in Best-of- N sampling or guide selection during Monte Carlo Tree Search (MCTS).

Architecture Flow Diagram

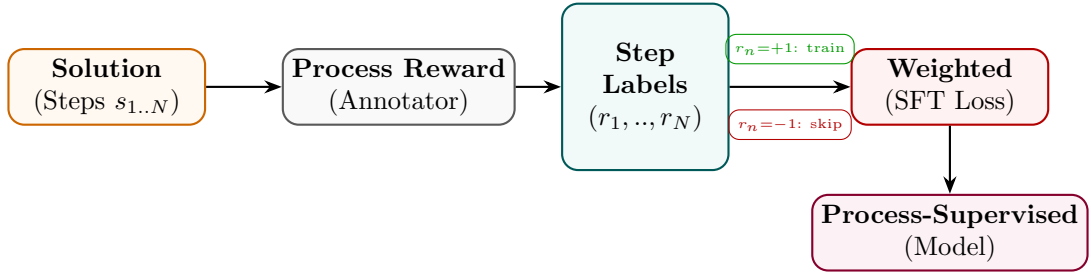


Figure 10: Process supervision: a step annotator labels each reasoning step; only correct steps contribute to the SFT loss.

State Transition Table

Property	Before	After
Supervision granularity	Final answer only	Per-step labels
Error signal	Outcome-level	Step-level
Error localisation	None	Identifies which step failed
Annotation cost	Low	High (step-level labelling)
Reasoning quality	Outcome-driven	Step-correctness driven

Table 10: State properties before and after process supervision.

Part III

Parameter-Efficient Fine-Tuning (PEFT)

Low-Rank Adaptation (LoRA)

Mathematical Foundation

LoRA represents the weight update matrix $\Delta W \in \mathbb{R}^{k \times d}$ for a linear projection layer as the product of two low-rank matrices $B \in \mathbb{R}^{k \times r}$ and $A \in \mathbb{R}^{r \times d}$, where the rank satisfies $r \ll \min(d, k)$:

$$h = W_0 x + \Delta W x = W_0 x + \frac{\alpha}{r} B A x \quad (35)$$

where α is a constant scaling parameter. The base weight matrix $W_0 \in \mathbb{R}^{k \times d}$ is frozen; only B and A are updated. Under a loss $\mathcal{L}$, the gradient propagation rules using the chain rule are derived as follows: Let $z = Ax \in \mathbb{R}^r$ be the intermediate bottleneck activation. Then $h = W_0 x + \frac{\alpha}{r} B z$. The gradient with respect to output activation is $\nabla_h \mathcal{L} \in \mathbb{R}^{k \times 1}$ (represented as a column vector). The gradient with respect to the intermediate bottleneck representation z :

$$\nabla_z \mathcal{L} = \frac{\alpha}{r} B^\top \nabla_h \mathcal{L} \in \mathbb{R}^{r \times 1} \quad (36)$$

The gradient with respect to the matrix B :

$$\nabla_B \mathcal{L} = \frac{\alpha}{r} \nabla_h \mathcal{L} z^\top = \frac{\alpha}{r} \nabla_h \mathcal{L} x^\top A^\top \in \mathbb{R}^{k \times r} \quad (37)$$

The gradient with respect to the matrix A :

$$\nabla_A \mathcal{L} = \nabla_z \mathcal{L} x^\top = \frac{\alpha}{r} B^\top \nabla_h \mathcal{L} x^\top \in \mathbb{R}^{r \times d} \quad (38)$$

To eliminate latency at deployment, the weights are merged directly into the base model:

$$W_{\text{merged}} = W_0 + \frac{\alpha}{r} B A \quad (39)$$

Architecture Flow Diagram

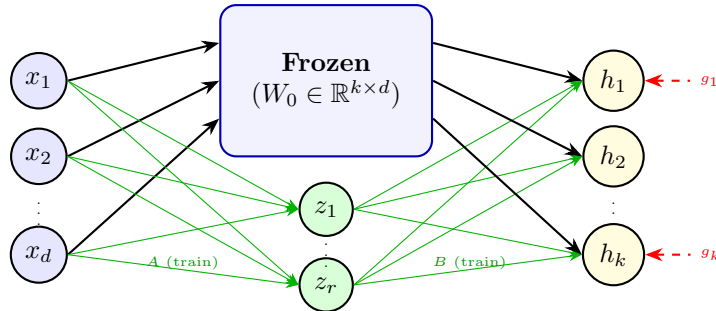


Figure 11: LoRA: input flows through frozen W_0 and trainable low-rank path $A \rightarrow B$; outputs are summed.

State Transition Table

Property	Before	After
Base weights W_0	requires_grad = True	Frozen (False)
Trainable params	$k \times d$	$r(k + d) \ll kd$
Matrix B	Does not exist	$\mathbb{R}^{k \times r}$, init = 0
Matrix A	Does not exist	$\mathbb{R}^{r \times d}$, init $\sim \mathcal{N}$
Inference overhead	None	Extra BA multiply (or merged)

Table 11: State properties before and after LoRA injection.

Memory Complexity Analysis

In this section, we analyze the memory footprint of the optimizer states. We compare full fine-tuning of a base weight matrix $W_0 \in \mathbb{R}^{k \times d}$ against low-rank training of A and B . Let $N_{\text{base}} = k \cdot d$ be the number of base parameters, and $N_{\text{LoRA}} = r(k + d)$ be the number of LoRA parameters.

AdamW tracks the following state elements for each trainable parameter:

1. **Parameter value copy** (master weights in mixed-precision): 4 bytes.
2. **First moment vector m (momentum)**: 4 bytes.
3. **Second moment vector v (variance)**: 4 bytes.

This results in $M_{\text{opt}} = 12$ bytes of optimizer state memory overhead per trainable parameter.

The memory saving ratio Φ is defined as:

$$\Phi = \frac{M_{\text{LoRA}}}{M_{\text{Full}}} = \frac{r(k + d)}{k \cdot d} = r \left(\frac{1}{k} + \frac{1}{d} \right) \quad (40)$$

Assuming a standard linear layer in a transformer block where $k = d = 4096$, and adapter rank $r = 8$:

$$\Phi = 8 \left(\frac{1}{4096} + \frac{1}{4096} \right) = \frac{16}{4096} = \frac{1}{256} \approx 0.39\% \quad (41)$$

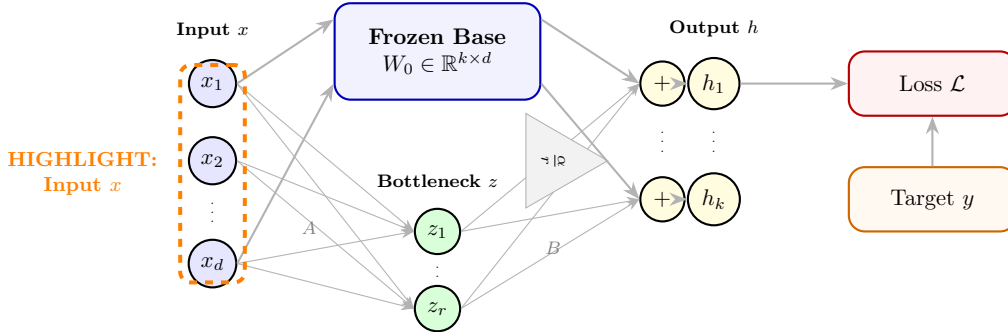
This indicates a **256-fold reduction** (a decrease of 99.61%) in the memory required to store optimizer states for this weight matrix.

Detailed Operational Analysis of LoRA Variables

To understand the execution of LoRA at a granular level, we detail the mathematical nature of each variable, how its values are derived in reality, its child-level control dial mental model, and its physical mechanism at the individual neuron level. We provide a full neuron-level highlight diagram for each variable.

1. Input Activation Vector (x):

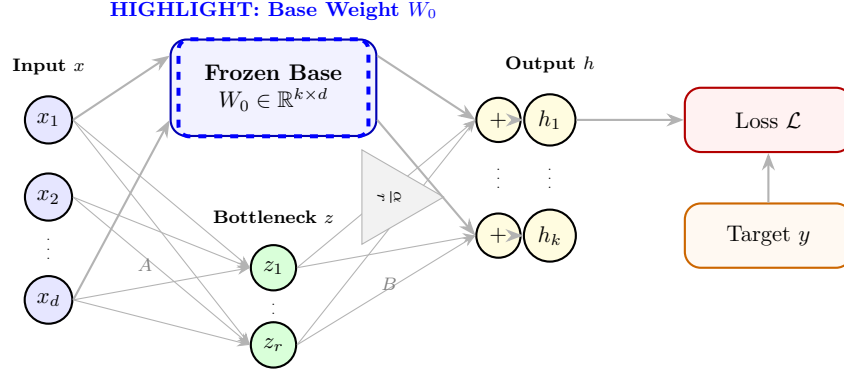
- **Formula:** x (output of previous layer)
- **Neuron Highlight Diagram:**



- **Mental Model:** Think of this vector as a list of positions for a row of thousands of physical dials. When the model processes a word, it adjusts these dials to specific numbers (e.g., dial 1 is at 4.2, dial 2 is at -1.5) to capture the meaning of the word. The model does not think in words; it thinks in these thousands of dial settings.
- **Neuron-Level Physical Explanation:** A single neuron in the current layer receives x as a set of electrical signals from the neurons of the previous layer. Each element x_i represents the activation level (firing rate) of a previous neuron after its activation function (such as ReLU or GELU). The dendrites of the current neuron receive these signals.
- **Mathematical Representation:** $x \in \mathbb{R}^{d \times 1}$
- **Code Representation:** `x` (a PyTorch tensor of shape `(batch_size, seq_len, d)`)
- **Relationships:** Fed into the base projection $W_0 x$ and compressed to $z = Ax$.

2. Frozen Base Weight Matrix (W_0):

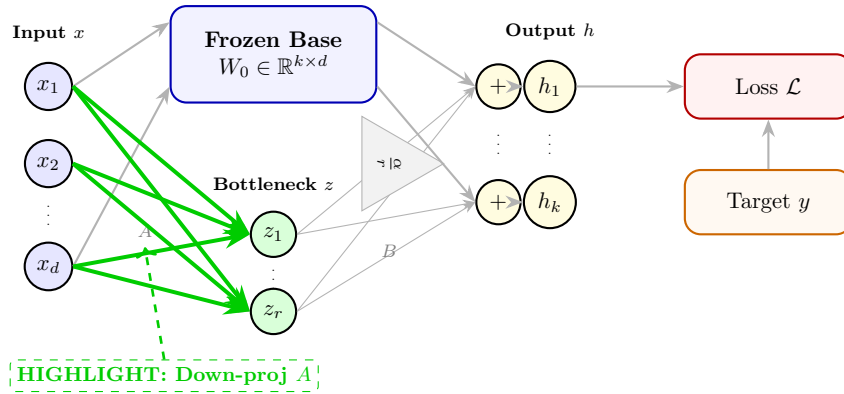
- **Formula:** W_0
- **Neuron Highlight Diagram:**



- **Mental Model:** This is a giant, permanent circuit board inside the control room. The wires are soldered in place and cannot be moved (frozen). They take the positions of the incoming dials (x), mix them together according to the permanent rules, and output a new set of general dial positions.
- **Neuron-Level Physical Explanation:** A single neuron's base connections are represented by a row of W_0 . The neuron performs a dot product: it multiplies each incoming signal x_i by the strength of its corresponding synaptic connection $W_{0,ji}$ and sums them up: $y_j = \sum_i W_{0,ji}x_i$. During LoRA, these synaptic weights are frozen and cannot change, meaning their chemical/electrical conductance is locked.
- **Mathematical Representation:** $W_0 \in \mathbb{R}^{k \times d}$
- **Code Representation:** `W_0` (a PyTorch tensor with `W_0.requires_grad = False` to disable gradient computation)
- **Relationships:** Multiplied by the input to compute the baseline representation: $h_{\text{base}} = W_0x$.

3. Down-projection Adapter (A):

- **Formula:** A
- **Neuron Highlight Diagram:**

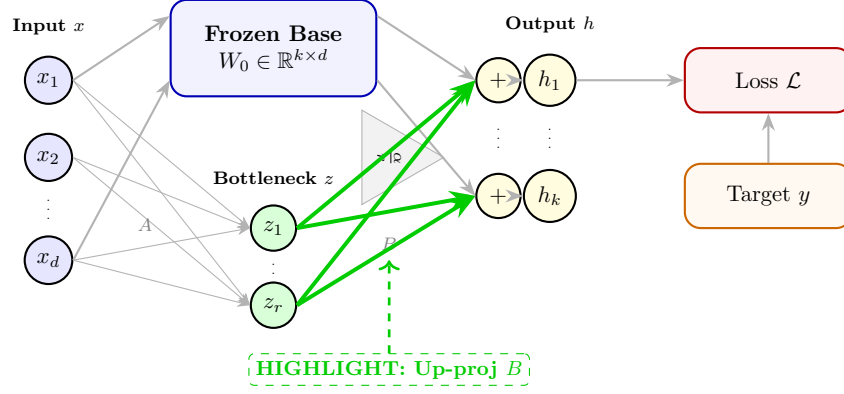


- **Mental Model:** This is a smaller circuit board that acts as a compression funnel. It looks at all the thousands of input dials and summarizes them into just a few dials (e.g., summarizing 4096 dials into just 8 dials). It only listens to the dial changes that are relevant to the new task we are teaching the model.
- **Neuron-Level Physical Explanation:** Inside the adapter path, a single neuron in the bottleneck layer corresponds to a row of matrix A . This neuron listens to all input signals x_i , multiplies them by its trainable weights A_{mi} , and sums them up: $z_m = \sum_i A_{mi}x_i$. This bottleneck neuron acts as a feature detector that extracts a specific combination of input signals.

- **Mathematical Representation:** $A \in \mathbb{R}^{r \times d}$
- **Code Representation:** A (initialized as `nn.Linear(d, r, bias=False)` with weight values drawn from a Gaussian distribution: `nn.init.kaiming_uniform_(A.weight, a=math.sqrt(5))`)
- **Relationships:** Compresses x into the bottleneck space via $z = Ax$.

4. Up-projection Adapter (B):

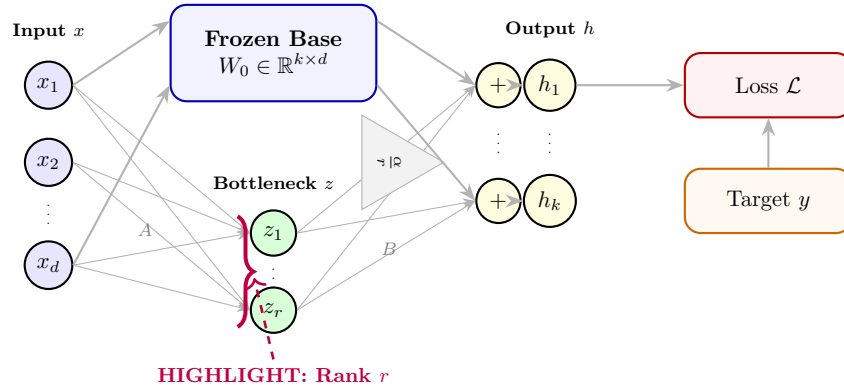
- **Formula:** B
- **Neuron Highlight Diagram:**



- **Mental Model:** This is another small circuit board that acts as an expansion nozzle. It takes the tiny summary from the funnel (our 8 dials) and expands it back into thousands of dials, showing how to nudge the final output dials to fit the new task.
- **Neuron-Level Physical Explanation:** A single output neuron receives contributions from the bottleneck neurons. The connection strength from bottleneck neuron m to output neuron j is the trainable weight B_{jm} . The output neuron sums the weighted signals from all bottleneck neurons: $u_j = \sum_m B_{jm} z_m$, scaling up the low-dimensional task features.
- **Mathematical Representation:** $B \in \mathbb{R}^{k \times r}$
- **Code Representation:** B (initialized as `nn.Linear(r, k, bias=False)` with weight values set to all zeros: `nn.init.zeros_(B.weight)`)
- **Relationships:** Expands the bottleneck activations via Bz , scaled by $\frac{\alpha}{r}$.

5. Bottleneck Rank (r):

- **Formula:** r
- **Neuron Highlight Diagram:**

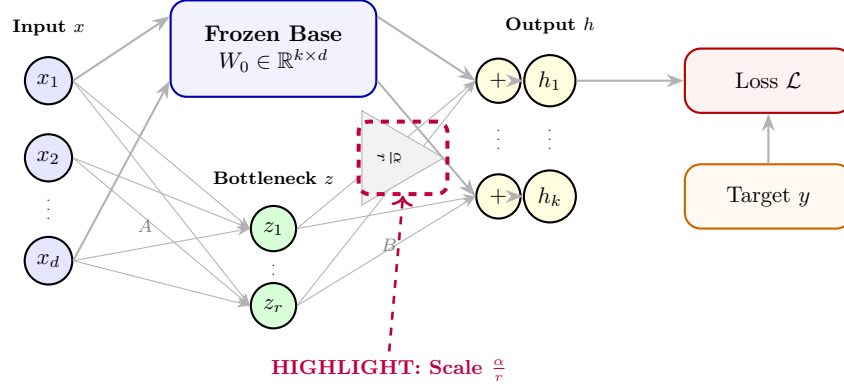


- **Mental Model:** This is the exact number of summary dials inside the bottleneck. If $r = 8$, it means we are forcing the model to summarize all its knowledge into just 8 dials. A smaller number makes the funnel tighter, forcing the model to only keep the most important summaries.

- **Neuron-Level Physical Explanation:** This is the exact number of neurons present in the bottleneck layer of the adapter. Selecting $r = 8$ means there are exactly 8 neurons in this intermediate layer. Each of these 8 neurons acts as a specialized task detector.
- **Mathematical Representation:** $r \in \mathbb{Z}^+$
- **Code Representation:** `r` (passed as an integer configuration parameter, e.g., `lora_config = LoraConfig(r=8)`)
- **Relationships:** Determines the inner dimension of matrices A and B , and acts as a scaling denominator in $\frac{\alpha}{r}$.

6. Scaling Constant (α):

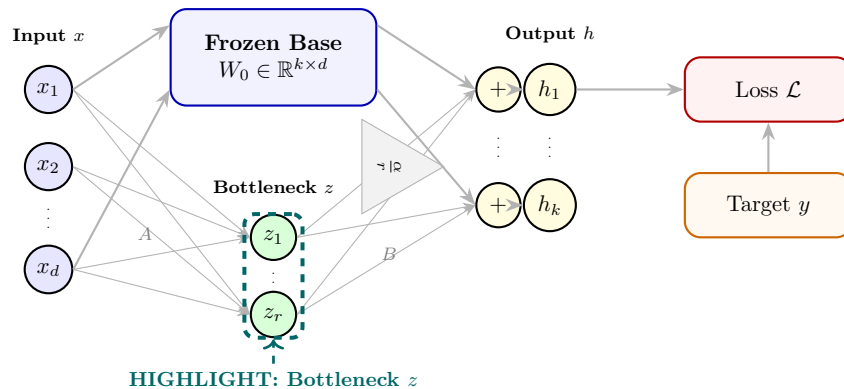
- **Formula:** α
- **Neuron Highlight Diagram:**



- **Mental Model:** This is a volume control knob. It decides how loudly the changes from the new adapters (A and B) should shout compared to the original, permanent circuit board (W_0). If we turn this knob up, the model listens more to its new training and less to its old pretrained knowledge.
- **Neuron-Level Physical Explanation:** At the neuron level, this acts as a synaptic gain control. The signal coming from the bottleneck via the adapter pathway is amplified or attenuated by a constant scale factor of $\frac{\alpha}{r}$ before it is added to the base neuron's output. This behaves like a biological neuromodulator that globally scales the synaptic sensitivity of the adapter path.
- **Mathematical Representation:** $\alpha \in \mathbb{R}^+$
- **Code Representation:** `lora_alpha` (passed as a configuration parameter, e.g., `lora_config = LoraConfig(lora_alpha=...`)
- **Relationships:** Multiplies the adapter pathway output, yielding $\frac{\alpha}{r} Bz$.

7. Bottleneck Activation Vector (z):

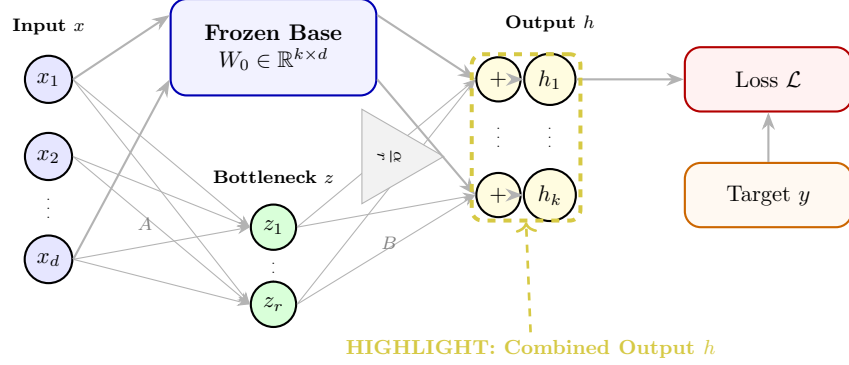
- **Formula:** $z = Ax$
- **Neuron Highlight Diagram:**



- **Mental Model:** These are the actual values of our few summary dials inside the bottleneck. It represents the compressed task-specific message as it flows from the compression funnel to the expansion nozzle.
- **Neuron-Level Physical Explanation:** Each element z_m is the electrical activation level (firing rate) of a single neuron in the bottleneck layer. It is calculated by that neuron summing its inputs: $z_m = \sum_i A_{mi}x_i$.
- **Mathematical Representation:** $z \in \mathbb{R}^{r \times 1}$
- **Code Representation:** $z = \text{F.linear}(x, A.\text{weight})$ (or $z = x @ A.\text{weight.t}()$)
- **Relationships:** Created by A , consumed by B .

8. Combined Output Vector (h):

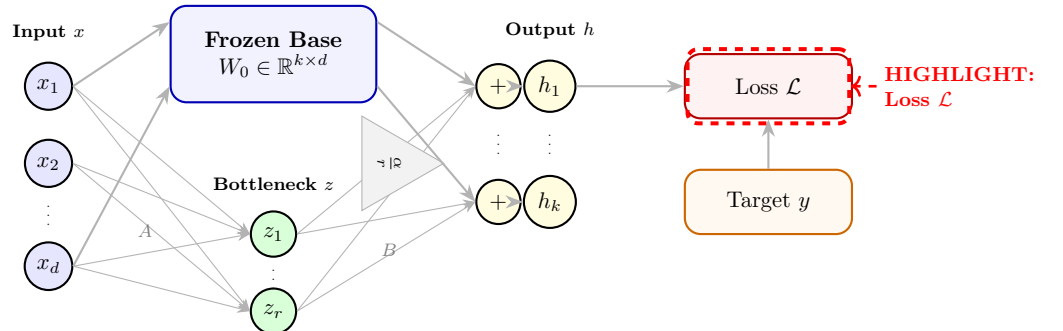
- **Formula:** $h = W_0x + \frac{\alpha}{r}Bz$
- **Neuron Highlight Diagram:**



- **Mental Model:** This is the final row of output dials. It is created by taking the general dial settings computed by the permanent circuit board (W_0) and nudging them slightly using the task-specific dial settings computed by our adapters.
- **Neuron-Level Physical Explanation:** A single output neuron j sums the signals from the frozen base connection path and the scaled trainable adapter path: $h_j = \sum_i W_{0,ji}x_i + \frac{\alpha}{r} \sum_m B_{jm}z_m$. The neuron acts as a physical summing junction, integrating the general-purpose signal and the specialized task signal.
- **Mathematical Representation:** $h \in \mathbb{R}^{k \times 1}$
- **Code Representation:** $h = \text{base_layer}(x) + (\text{alpha} / r) * B(z)$
- **Relationships:** Fed as input to subsequent layers of the network.

9. Scalar Loss Value ($\mathcal{L}$):

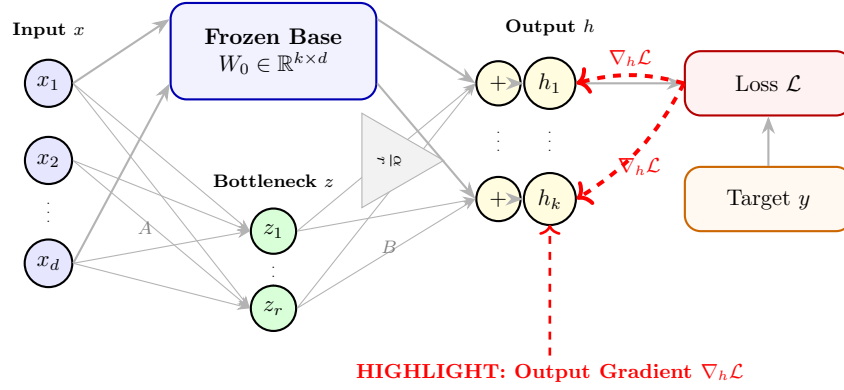
- **Formula:** $\mathcal{L}$ (e.g. Cross-Entropy loss)
- **Neuron Highlight Diagram:**



- **Mental Model:** This is a single scorecard number. It tells us how far off the final output dials are from what they should be. If the score is high, it means the model is making bad guesses. The goal of training is to adjust the new dials until the scorecard reads zero.
- **Neuron-Level Physical Explanation:** The loss $\mathcal{L}$ measures how far the firing rates of the output neurons in the final layer differ from the desired target firing rates. The target represents the correct next word, and the error signal is generated by comparing the target with the actual outputs.
- **Mathematical Representation:** $\mathcal{L} \in \mathbb{R}$
- **Code Representation:** `loss = F.cross_entropy(logits, targets)`
- **Relationships:** Traversed in reverse during backpropagation; all gradients are derivatives of $\mathcal{L}$.

10. Output Layer Gradient ($\nabla_h \mathcal{L}$):

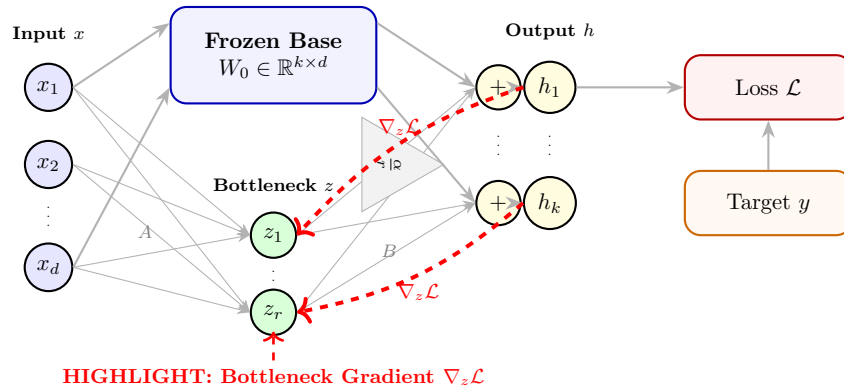
- **Formula:** $\nabla_h \mathcal{L} = \frac{\partial \mathcal{L}}{\partial h}$
- **Neuron Highlight Diagram:**



- **Mental Model:** This is a set of correction instructions for the final output dials. It says: "dial 1 needs to turn clockwise by 0.2, and dial 2 needs to turn counter-clockwise by 1.5" to lower the error score.
- **Neuron-Level Physical Explanation:** For a single output neuron j , the element $(\nabla_h \mathcal{L})_j = \frac{\partial \mathcal{L}}{\partial h_j}$ represents the error feedback signal. It tells the neuron whether it fired too strongly (positive gradient) or too weakly (negative gradient) and by how much it needs to adjust its total output to minimize the error.
- **Mathematical Representation:** $\nabla_h \mathcal{L} \in \mathbb{R}^{k \times 1}$
- **Code Representation:** automatically tracked by PyTorch's `loss.backward()`, represented as the grad tensor associated with output `h.grad`
- **Relationships:** Directly drives the gradient computation for B and z .

11. Bottleneck Layer Gradient ($\nabla_z \mathcal{L}$):

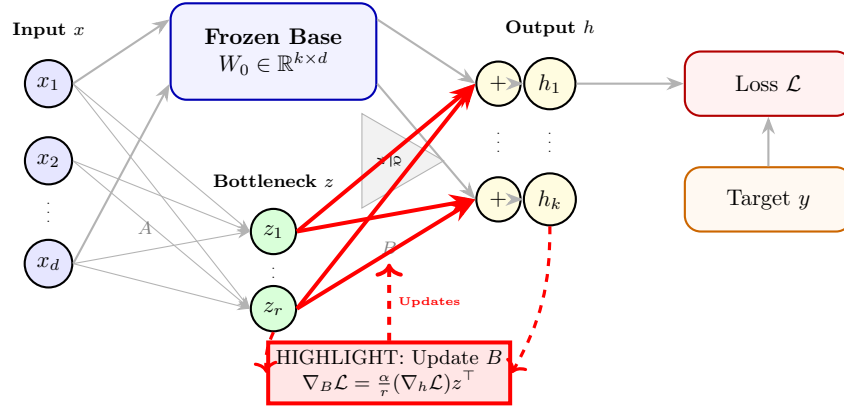
- **Formula:** $\nabla_z \mathcal{L} = \frac{\alpha}{r} B^\top \nabla_h \mathcal{L}$
- **Neuron Highlight Diagram:**



- **Mental Model:** This translates the correction instructions from the output room back down into the bottleneck. It tells the summary dials inside the funnel how they need to adjust to fix the output dials.
- **Neuron-Level Physical Explanation:** For a bottleneck neuron m , this is the backpropagated error signal. It is computed by summing the error feedback signals from all output neurons, weighted by their connections B_{jm} and scaled: $(\nabla_z \mathcal{L})_m = \frac{\alpha}{r} \sum_j B_{jm} (\nabla_h \mathcal{L})_j$. It tells the bottleneck neuron how its activation should have changed.
- **Mathematical Representation:** $\nabla_z \mathcal{L} \in \mathbb{R}^{r \times 1}$
- **Code Representation:** `grad_z = (alpha / r) * torch.matmul(grad_output, B.weight)`
- **Relationships:** Drives the gradient computation for A .

12. Update Gradient for Matrix B ($\nabla_B \mathcal{L}$):

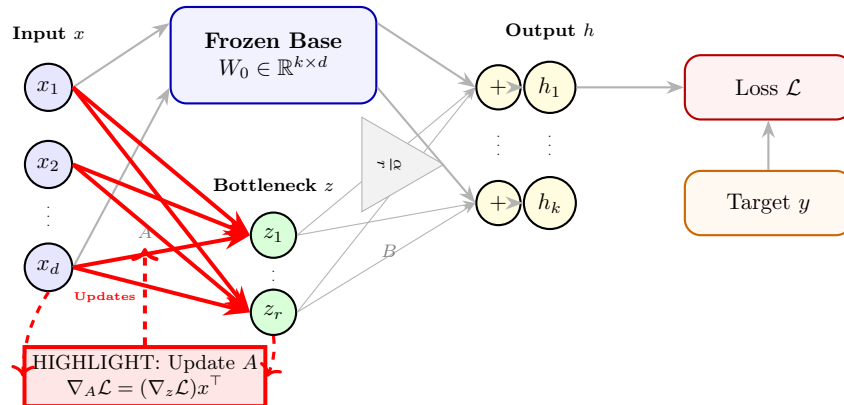
- **Formula:** $\nabla_B \mathcal{L} = \frac{\alpha}{r} (\nabla_h \mathcal{L}) z^\top$
- **Neuron Highlight Diagram:**



- **Mental Model:** This is the instruction sheet showing how to rewire the dials of the expansion nozzle (B) so that they do a better job of translating the summary back to the output dials.
- **Neuron-Level Physical Explanation:** For the synapse connecting bottleneck neuron m to output neuron j , the gradient is calculated as $(\nabla_B \mathcal{L})_{jm} = \frac{\alpha}{r} (\nabla_h \mathcal{L})_j z_m$. This follows Hebbian learning: the strength of the update is proportional to the product of the pre-synaptic activation (z_m) and the post-synaptic error signal ($(\nabla_h \mathcal{L})_j$).
- **Mathematical Representation:** $\nabla_B \mathcal{L} \in \mathbb{R}^{k \times r}$
- **Code Representation:** `B.weight.grad = (alpha / r) * torch.matmul(grad_output.t(), z)`
- **Relationships:** Used by the optimizer to update the values of B .

13. Update Gradient for Matrix A ($\nabla_A \mathcal{L}$):

- **Formula:** $\nabla_A \mathcal{L} = (\nabla_z \mathcal{L}) x^\top$
- **Neuron Highlight Diagram:**



- **Mental Model:** This is the instruction sheet showing how to rewire the dials of the compression funnel (A) so that they do a better job of compressing the input dials into the summary.
- **Neuron-Level Physical Explanation:** For the synapse connecting input neuron i to bottleneck neuron m , the gradient is $(\nabla_A \mathcal{L})_{mi} = (\nabla_z \mathcal{L})_m x_i$. This also follows Hebbian learning: it multiplies the pre-synaptic input signal (x_i) by the post-synaptic bottleneck error signal $((\nabla_z \mathcal{L})_m)$ to determine how much the synapse should strengthen or weaken.
- **Mathematical Representation:** $\nabla_A \mathcal{L} \in \mathbb{R}^{r \times d}$
- **Code Representation:** `A.weight.grad = torch.matmul(grad_z.t(), x)`
- **Relationships:** Used by the optimizer to update the values of A .

The Physical and Philosophical Foundations of LoRA

Philosophical Foundations: Occam’s Razor and the Manifold Hypothesis

The success of Low-Rank Adaptation (LoRA) is not merely an empirical coincidence, but is deeply rooted in two foundational philosophical principles of complexity. We analyze each principle through a dialectic framework of Arguments, Counter-Arguments, and Rebuttals:

1. The Manifold Hypothesis

- **Argument:** This hypothesis posits that high-dimensional data (such as the activations in a neural network) actually lies on a much lower-dimensional, highly structured manifold. Pretraining constructs this manifold by mapping the topological structure of natural language.

In the context of Large Language Models, the embedding space has an extrinsic dimension of $d = 4096$ (e.g., $d_{\text{model}} = 4096$). However, the set of all grammatically correct and semantically coherent sentences does not fill this space uniformly; instead, it is confined to a lower-dimensional manifold. When we adapt the model to a downstream task (e.g., teaching it Python syntax or chat style), we do not need to construct a new manifold. We only need to compute a small trajectory or translation vector along the existing surface. This translation vector corresponds to a tangent vector in the manifold’s tangent bundle. By confining our search to this tangent space, we avoid distorting the global topology of the language manifold, preventing catastrophic forgetting.

- **Counter-Argument:** If the downstream task represents an extreme domain shift (e.g., shifting from English literature to highly specialized biochemistry nomenclature or low-level assembly code), the target representations may lie completely outside the manifold mapped during pretraining. In such cases, the translation vector cannot lie along the existing manifold surface, and restricting the updates to a low-rank tangent space will lead to underfitting. The model would fail to acquire the new features because the required coordinates do not exist in the pretrained column space.
- **Rebuttal / Synthesis:** While severe domain shifts can strain low-rank constraints, empirical evidence shows that even highly specialized knowledge maps back to general-purpose linguistic structures (syntax, logic, context). Furthermore, if the target lies slightly off-manifold, a small rank r (e.g., 8 to 16) is still sufficient to project the input into the nearest point on the task manifold. If needed, the user can scale the rank r to increase the degrees of freedom, which mathematically expands the coordinate projection space, allowing the model to capture features that lie slightly outside the pretraining manifold while still regularizing the update to prevent arbitrary drift.

2. Occam’s Razor (Lex Parsimoniae)

- **Argument:** Plurality should not be posited without necessity. Full fine-tuning searches an extremely large, redundant parameter space ($\mathbb{R}^{k \times d}$), which contains millions of redundant degrees of freedom. This massive parameter space allows the model to find arbitrary, highly complex fitting functions that overfit the downstream training data and overwrite pretrained general knowledge.

By restricting the learning search to a compressed subspace ($\mathbb{R}^{r \times (d+k)}$), LoRA acts as a mathematical regularization agent. It forces the model to find the simplest, most parsimonious path of adaptation. This acts as a geometric constraint (projection regularization), restricting updates to only the principal directions of adaptation, thereby preserving the model’s core foundational capabilities.

- **Counter-Argument:** By restricting the parameters to a low-rank product BA , we might force the model to find an oversimplified adaptation path that fails to capture the subtle, non-linear correlations required for complex reasoning tasks. Furthermore, Occam’s Razor assumes that simpler models generalize better, but in deep learning, the phenomenon of “double descent” shows that highly overparameterized models can generalize better than simpler ones by finding smooth interpolating functions in high-dimensional spaces.

- **Rebuttal / Synthesis:** Double descent occurs when the entire network is overparameterized and trained on vast, diverse datasets where it can interpolate smoothly. However, in downstream fine-tuning, the dataset is small and highly focused. In this regime, overparameterization leads to catastrophic forgetting and representation collapse. Constraining the updates to a low-rank subspace restricts the model from modifying the general-purpose features, while allowing sufficient capacity to learn the task. Thus, LoRA combines the benefits of high-dimensional pretraining (smooth interpolation on general data) with low-dimensional adaptation (strict regularization on task-specific data).

Internal Mechanics and Representation Space Geometry

To understand why LoRA works under the hood, we must analyze the geometry of representation space and the intrinsic dimension of neural networks:

- **Intrinsic vs. Extrinsic Dimension:** A pretrained model has a very high extrinsic dimension (the number of weights $W_0 \in \mathbb{R}^{k \times d}$). However, empirical work by Aghajanyan et al. (2020) demonstrated that the parameter space of pretrained language models has a much lower *intrinsic dimension*. That is, the directions in weight space that significantly affect the loss are highly concentrated on a low-dimensional subspace. This is why we can optimize a tiny fraction of parameters and achieve the same or better task performance.
- **Subspace Alignment and Trajectory Geometry:** The activation vectors x represent coordinates on the manifold. The base weight matrix W_0 maps these coordinates to the next representation space. The update matrix $\Delta W = \frac{\alpha}{r}BA$ acts as a linear projection operator that projects the activations into a tiny subspace of dimension r via A , rotates/scales them in this bottleneck space, and then projects them back to the output space via B . This operation restricts the adaptation to a set of r directional vectors. Instead of shifting the entire coordinate frame, the adapter only redirects the activation trajectory along the principal directions relevant to the downstream task.

Geometrically, the column space of ΔW is spanned by the columns of B , which is an r -dimensional subspace of the output space $\mathbb{R}^k$. The row space of ΔW is spanned by the rows of A , which is an r -dimensional subspace of the input space $\mathbb{R}^d$. Thus, the adaptation updates are strictly constrained to these coordinates, preserving all other dimensions from degradation.

Visualizing the Calculations

We can visualize how the information bottleneck constrains the updates in the representation space during the forward and backward passes:

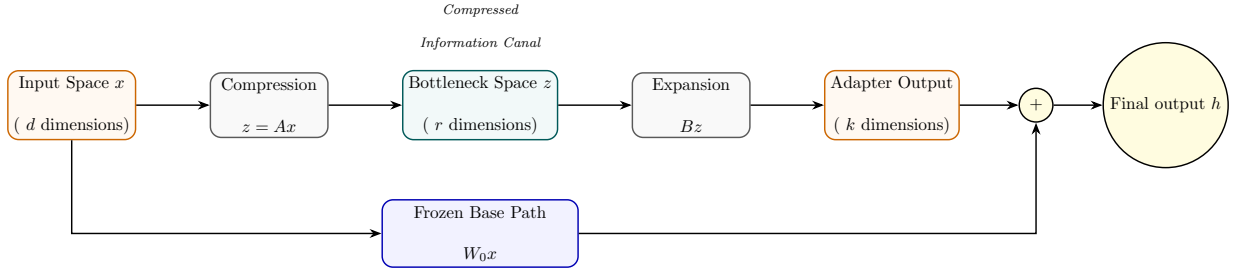


Figure 12: Visualization of the information bottleneck: high-dimensional input is projected into a low-dimensional canal where task-specific tuning forces are applied, then projected back.

- **Forward Pass Compression:** The activation vector $x \in \mathbb{R}^{d \times 1}$ is projected down through a narrow bottleneck of rank r . Since $r \ll \min(d, k)$, the projection filters out dimensions irrelevant to the current task, creating a compressed task representation $z \in \mathbb{R}^{r \times 1}$.
- **Backward Pass Gradient Flow:** Error gradients $\nabla_h \mathcal{L}$ flow backwards. Because W_0 is frozen and lacks a gradient buffer, no gradient update calculations occur on the base weight matrix. Instead, the gradients flow down the low-rank adapters B and A , computing updates specifically for the low-rank subspace and avoiding parameter updates across the whole network.

The Physical and Philosophical Foundations of LoRA

Philosophical Foundations: Occam's Razor and the Manifold Hypothesis

The success of Low-Rank Adaptation (LoRA) is not merely an empirical coincidence, but is deeply rooted in two foundational philosophical principles of complexity. We analyze each principle through a dialectic framework of Arguments, Counter-Arguments, and Rebuttals:

1. The Manifold Hypothesis

- **Argument:** This hypothesis posits that high-dimensional data (such as the activations in a neural network) actually lies on a much lower-dimensional, highly structured manifold. Pretraining constructs this manifold by mapping the topological structure of natural language.

In the context of Large Language Models, the embedding space has an extrinsic dimension of $d = 4096$ (e.g., $d_{\text{model}} = 4096$). However, the set of all grammatically correct and semantically coherent sentences does not fill this space uniformly; instead, it is confined to a lower-dimensional manifold. When we adapt the model to a downstream task (e.g., teaching it Python syntax or chat style), we do not need to construct a new manifold. We only need to compute a small trajectory or translation vector along the existing surface. This translation vector corresponds to a tangent vector in the manifold’s tangent bundle. By confining our search to this tangent space, we avoid distorting the global topology of the language manifold, preventing catastrophic forgetting.

- **Counter-Argument:** If the downstream task represents an extreme domain shift (e.g., shifting from English literature to highly specialized biochemistry nomenclature or low-level assembly code), the target representations may lie completely outside the manifold mapped during pretraining. In such cases, the translation vector cannot lie along the existing manifold surface, and restricting the updates to a low-rank tangent space will lead to underfitting. The model would fail to acquire the new features because the required coordinates do not exist in the pretrained column space.
- **Rebuttal / Synthesis:** While severe domain shifts can strain low-rank constraints, empirical evidence shows that even highly specialized knowledge maps back to general-purpose linguistic structures (syntax, logic, context). Furthermore, if the target lies slightly off-manifold, a small rank r (e.g., 8 to 16) is still sufficient to project the input into the nearest point on the task manifold. If needed, the user can scale the rank r to increase the degrees of freedom, which mathematically expands the coordinate projection space, allowing the model to capture features that lie slightly outside the pretraining manifold while still regularizing the update to prevent arbitrary drift.

2. Occam’s Razor (Lex Parsimoniae)

- **Argument:** Plurality should not be posited without necessity. Full fine-tuning searches an extremely large, redundant parameter space ($\mathbb{R}^{k \times d}$), which contains millions of redundant degrees of freedom. This massive parameter space allows the model to find arbitrary, highly complex fitting functions that overfit the downstream training data and overwrite pretrained general knowledge.

By restricting the learning search to a compressed subspace ($\mathbb{R}^{r \times (d+k)}$), LoRA acts as a mathematical regularization agent. It forces the model to find the simplest, most parsimonious path of adaptation. This acts as a geometric constraint (projection regularization), restricting updates to only the principal directions of adaptation, thereby preserving the model’s core foundational capabilities.

- **Counter-Argument:** By restricting the parameters to a low-rank product BA , we might force the model to find an oversimplified adaptation path that fails to capture the subtle, non-linear correlations required for complex reasoning tasks. Furthermore, Occam’s Razor assumes that simpler models generalize better, but in deep learning, the phenomenon of “double descent” shows that highly overparameterized models can generalize better than simpler ones by finding smooth interpolating functions in high-dimensional spaces.
- **Rebuttal / Synthesis:** Double descent occurs when the entire network is overparameterized and trained on vast, diverse datasets where it can interpolate smoothly. However, in downstream fine-tuning, the dataset is small and highly focused. In this regime, overparameterization leads to catastrophic forgetting and representation collapse. Constraining the updates to a low-rank subspace restricts the model from modifying the general-purpose features, while allowing sufficient capacity to learn the task. Thus, LoRA combines the benefits of high-dimensional pretraining (smooth interpolation on general data) with low-dimensional adaptation (strict regularization on task-specific data).

Internal Mechanics and Representation Space Geometry

To understand why LoRA works under the hood, we must analyze the geometry of representation space and the intrinsic dimension of neural networks:

- **Intrinsic vs. Extrinsic Dimension:** A pretrained model has a very high extrinsic dimension (the number of weights $W_0 \in \mathbb{R}^{k \times d}$). However, empirical work by Aghajanyan et al. (2020) demonstrated that the parameter space of pretrained language models has a much lower *intrinsic dimension*. That is, the directions in weight space that significantly affect the loss are highly concentrated on a low-dimensional subspace. This is why we can optimize a tiny fraction of parameters and achieve the same or better task performance.

- **Subspace Alignment and Trajectory Geometry:** The activation vectors x represent coordinates on the manifold. The base weight matrix W_0 maps these coordinates to the next representation space. The update matrix $\Delta W = \frac{\alpha}{r}BA$ acts as a linear projection operator that projects the activations into a tiny subspace of dimension r via A , rotates/scales them in this bottleneck space, and then projects them back to the output space via B . This operation restricts the adaptation to a set of r directional vectors. Instead of shifting the entire coordinate frame, the adapter only redirects the activation trajectory along the principal directions relevant to the downstream task.

Geometrically, the column space of ΔW is spanned by the columns of B , which is an r -dimensional subspace of the output space $\mathbb{R}^k$. The row space of ΔW is spanned by the rows of A , which is an r -dimensional subspace of the input space $\mathbb{R}^d$. Thus, the adaptation updates are strictly constrained to these coordinates, preserving all other dimensions from degradation.

Visualizing the Calculations

We can visualize how the information bottleneck constrains the updates in the representation space during the forward and backward passes:

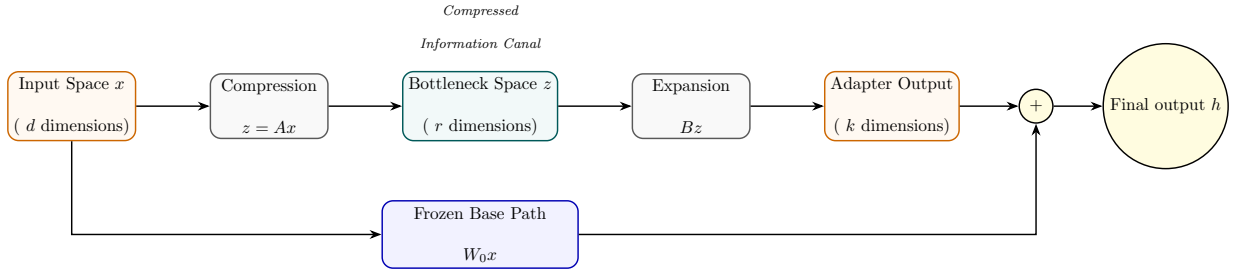


Figure 13: Visualization of the information bottleneck: high-dimensional input is projected into a low-dimensional canal where task-specific tuning forces are applied, then projected back.

- **Forward Pass Compression:** The activation vector $x \in \mathbb{R}^{d \times 1}$ is projected down through a narrow bottleneck of rank r . Since $r \ll \min(d, k)$, the projection filters out dimensions irrelevant to the current task, creating a compressed task representation $z \in \mathbb{R}^{r \times 1}$.
- **Backward Pass Gradient Flow:** Error gradients $\nabla_h \mathcal{L}$ flow backwards. Because W_0 is frozen and lacks a gradient buffer, no gradient update calculations occur on the base weight matrix. Instead, the gradients flow down the low-rank adapters B and A , computing updates specifically for the low-rank subspace and avoiding parameter updates across the whole network.

The Physical and Philosophical Foundations of LoRA

Philosophical Foundations: Occam’s Razor and the Manifold Hypothesis

The success of Low-Rank Adaptation (LoRA) is not merely an empirical coincidence, but is deeply rooted in two foundational philosophical principles of complexity. We analyze each principle through a dialectic framework of Arguments, Counter-Arguments, and Rebuttals:

1. The Manifold Hypothesis

- **Argument:** This hypothesis posits that high-dimensional data (such as the activations in a neural network) actually lies on a much lower-dimensional, highly structured manifold. Pretraining constructs this manifold by mapping the topological structure of natural language.

In the context of Large Language Models, the embedding space has an extrinsic dimension of $d = 4096$ (e.g., $d_{\text{model}} = 4096$). However, the set of all grammatically correct and semantically coherent sentences does not fill this space uniformly; instead, it is confined to a lower-dimensional manifold. When we adapt the model to a downstream task (e.g., teaching it Python syntax or chat style), we do not need to construct a new manifold. We only need to compute a small trajectory or translation vector along the existing surface. This translation vector corresponds to a tangent vector in the manifold’s tangent bundle. By confining our search to this tangent space, we avoid distorting the global topology of the language manifold, preventing catastrophic forgetting.

- **Counter-Argument:** If the downstream task represents an extreme domain shift (e.g., shifting from English literature to highly specialized biochemistry nomenclature or low-level assembly code), the target representations may lie completely outside the manifold mapped during pretraining. In such cases, the translation vector cannot lie along the existing manifold surface, and restricting the updates to a low-rank tangent space will lead to

underfitting. The model would fail to acquire the new features because the required coordinates do not exist in the pretrained column space.

- **Rebuttal / Synthesis:** While severe domain shifts can strain low-rank constraints, empirical evidence shows that even highly specialized knowledge maps back to general-purpose linguistic structures (syntax, logic, context). Furthermore, if the target lies slightly off-manifold, a small rank r (e.g., 8 to 16) is still sufficient to project the input into the nearest point on the task manifold. If needed, the user can scale the rank r to increase the degrees of freedom, which mathematically expands the coordinate projection space, allowing the model to capture features that lie slightly outside the pretraining manifold while still regularizing the update to prevent arbitrary drift.

2. Occam’s Razor (Lex Parsimoniae)

- **Argument:** Plurality should not be posited without necessity. Full fine-tuning searches an extremely large, redundant parameter space ($\mathbb{R}^{k \times d}$), which contains millions of redundant degrees of freedom. This massive parameter space allows the model to find arbitrary, highly complex fitting functions that overfit the downstream training data and overwrite pretrained general knowledge.

By restricting the learning search to a compressed subspace ($\mathbb{R}^{r \times (d+k)}$), LoRA acts as a mathematical regularization agent. It forces the model to find the simplest, most parsimonious path of adaptation. This acts as a geometric constraint (projection regularization), restricting updates to only the principal directions of adaptation, thereby preserving the model’s core foundational capabilities.

- **Counter-Argument:** By restricting the parameters to a low-rank product BA , we might force the model to find an oversimplified adaptation path that fails to capture the subtle, non-linear correlations required for complex reasoning tasks. Furthermore, Occam’s Razor assumes that simpler models generalize better, but in deep learning, the phenomenon of "double descent" shows that highly overparameterized models can generalize better than simpler ones by finding smooth interpolating functions in high-dimensional spaces.
- **Rebuttal / Synthesis:** Double descent occurs when the entire network is overparameterized and trained on vast, diverse datasets where it can interpolate smoothly. However, in downstream fine-tuning, the dataset is small and highly focused. In this regime, overparameterization leads to catastrophic forgetting and representation collapse. Constraining the updates to a low-rank subspace restricts the model from modifying the general-purpose features, while allowing sufficient capacity to learn the task. Thus, LoRA combines the benefits of high-dimensional pretraining (smooth interpolation on general data) with low-dimensional adaptation (strict regularization on task-specific data).

Internal Mechanics and Representation Space Geometry

To understand why LoRA works under the hood, we must analyze the geometry of representation space and the intrinsic dimension of neural networks:

- **Intrinsic vs. Extrinsic Dimension:** A pretrained model has a very high extrinsic dimension (the number of weights $W_0 \in \mathbb{R}^{k \times d}$). However, empirical work by Aghajanyan et al. (2020) demonstrated that the parameter space of pretrained language models has a much lower *intrinsic dimension*. That is, the directions in weight space that significantly affect the loss are highly concentrated on a low-dimensional subspace. This is why we can optimize a tiny fraction of parameters and achieve the same or better task performance.
- **Subspace Alignment and Trajectory Geometry:** The activation vectors x represent coordinates on the manifold. The base weight matrix W_0 maps these coordinates to the next representation space. The update matrix $\Delta W = \frac{\alpha}{r}BA$ acts as a linear projection operator that projects the activations into a tiny subspace of dimension r via A , rotates/scales them in this bottleneck space, and then projects them back to the output space via B . This operation restricts the adaptation to a set of r directional vectors. Instead of shifting the entire coordinate frame, the adapter only redirects the activation trajectory along the principal directions relevant to the downstream task.

Geometrically, the column space of ΔW is spanned by the columns of B , which is an r -dimensional subspace of the output space $\mathbb{R}^k$. The row space of ΔW is spanned by the rows of A , which is an r -dimensional subspace of the input space $\mathbb{R}^d$. Thus, the adaptation updates are strictly constrained to these coordinates, preserving all other dimensions from degradation.

Visualizing the Calculations

We can visualize how the information bottleneck constrains the updates in the representation space during the forward and backward passes:

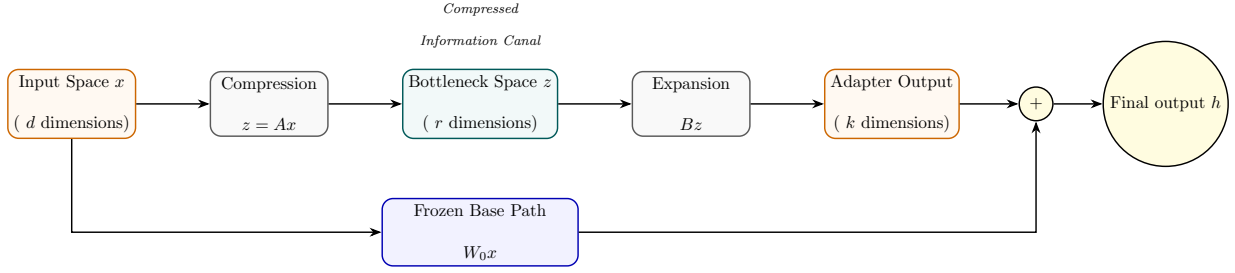


Figure 14: Visualization of the information bottleneck: high-dimensional input is projected into a low-dimensional canal where task-specific tuning forces are applied, then projected back.

- **Forward Pass Compression:** The activation vector $x \in \mathbb{R}^{d \times 1}$ is projected down through a narrow bottleneck of rank r . Since $r \ll \min(d, k)$, the projection filters out dimensions irrelevant to the current task, creating a compressed task representation $z \in \mathbb{R}^{r \times 1}$.
- **Backward Pass Gradient Flow:** Error gradients $\nabla_h \mathcal{L}$ flow backwards. Because W_0 is frozen and lacks a gradient buffer, no gradient update calculations occur on the base weight matrix. Instead, the gradients flow down the low-rank adapters B and A , computing updates specifically for the low-rank subspace and avoiding parameter updates across the whole network.

The Physical and Philosophical Foundations of LoRA

Philosophical Foundations: Occam’s Razor and the Manifold Hypothesis

The success of Low-Rank Adaptation (LoRA) is not merely an empirical coincidence, but is deeply rooted in two foundational philosophical principles of complexity. We analyze each principle through a dialectic framework of Arguments, Counter-Arguments, and Rebuttals:

1. The Manifold Hypothesis

- **Argument:** This hypothesis posits that high-dimensional data (such as the activations in a neural network) actually lies on a much lower-dimensional, highly structured manifold. Pretraining constructs this manifold by mapping the topological structure of natural language.

In the context of Large Language Models, the embedding space has an extrinsic dimension of $d = 4096$ (e.g., $d_{\text{model}} = 4096$). However, the set of all grammatically correct and semantically coherent sentences does not fill this space uniformly; instead, it is confined to a lower-dimensional manifold. When we adapt the model to a downstream task (e.g., teaching it Python syntax or chat style), we do not need to construct a new manifold. We only need to compute a small trajectory or translation vector along the existing surface. This translation vector corresponds to a tangent vector in the manifold’s tangent bundle. By confining our search to this tangent space, we avoid distorting the global topology of the language manifold, preventing catastrophic forgetting.

- **Counter-Argument:** If the downstream task represents an extreme domain shift (e.g., shifting from English literature to highly specialized biochemistry nomenclature or low-level assembly code), the target representations may lie completely outside the manifold mapped during pretraining. In such cases, the translation vector cannot lie along the existing manifold surface, and restricting the updates to a low-rank tangent space will lead to underfitting. The model would fail to acquire the new features because the required coordinates do not exist in the pretrained column space.
- **Rebuttal / Synthesis:** While severe domain shifts can strain low-rank constraints, empirical evidence shows that even highly specialized knowledge maps back to general-purpose linguistic structures (syntax, logic, context). Furthermore, if the target lies slightly off-manifold, a small rank r (e.g., 8 to 16) is still sufficient to project the input into the nearest point on the task manifold. If needed, the user can scale the rank r to increase the degrees of freedom, which mathematically expands the coordinate projection space, allowing the model to capture features that lie slightly outside the pretraining manifold while still regularizing the update to prevent arbitrary drift.

2. Occam’s Razor (Lex Parsimoniae)

- **Argument:** Plurality should not be posited without necessity. Full fine-tuning searches an extremely large, redundant parameter space ($\mathbb{R}^{k \times d}$), which contains millions of redundant degrees of freedom. This massive parameter space allows the model to find arbitrary, highly complex fitting functions that overfit the downstream training data and overwrite pretrained general knowledge.

By restricting the learning search to a compressed subspace ($\mathbb{R}^{r \times (d+k)}$), LoRA acts as a mathematical regularization agent. It forces the model to find the simplest, most parsimonious path of adaptation. This acts as a geometric constraint (projection regularization), restricting updates to only the principal directions of adaptation, thereby preserving the model’s core foundational capabilities.

- **Counter-Argument:** By restricting the parameters to a low-rank product BA , we might force the model to find an oversimplified adaptation path that fails to capture the subtle, non-linear correlations required for complex reasoning tasks. Furthermore, Occam’s Razor assumes that simpler models generalize better, but in deep learning, the phenomenon of “double descent” shows that highly overparameterized models can generalize better than simpler ones by finding smooth interpolating functions in high-dimensional spaces.
- **Rebuttal / Synthesis:** Double descent occurs when the entire network is overparameterized and trained on vast, diverse datasets where it can interpolate smoothly. However, in downstream fine-tuning, the dataset is small and highly focused. In this regime, overparameterization leads to catastrophic forgetting and representation collapse. Constraining the updates to a low-rank subspace restricts the model from modifying the general-purpose features, while allowing sufficient capacity to learn the task. Thus, LoRA combines the benefits of high-dimensional pretraining (smooth interpolation on general data) with low-dimensional adaptation (strict regularization on task-specific data).

Internal Mechanics and Representation Space Geometry

To understand why LoRA works under the hood, we must analyze the geometry of representation space and the intrinsic dimension of neural networks:

- **Intrinsic vs. Extrinsic Dimension:** A pretrained model has a very high extrinsic dimension (the number of weights $W_0 \in \mathbb{R}^{k \times d}$). However, empirical work by Aghajanyan et al. (2020) demonstrated that the parameter space of pretrained language models has a much lower *intrinsic dimension*. That is, the directions in weight space that significantly affect the loss are highly concentrated on a low-dimensional subspace. This is why we can optimize a tiny fraction of parameters and achieve the same or better task performance.
- **Subspace Alignment and Trajectory Geometry:** The activation vectors x represent coordinates on the manifold. The base weight matrix W_0 maps these coordinates to the next representation space. The update matrix $\Delta W = \frac{\alpha}{r}BA$ acts as a linear projection operator that projects the activations into a tiny subspace of dimension r via A , rotates/scales them in this bottleneck space, and then projects them back to the output space via B . This operation restricts the adaptation to a set of r directional vectors. Instead of shifting the entire coordinate frame, the adapter only redirects the activation trajectory along the principal directions relevant to the downstream task.

Geometrically, the column space of ΔW is spanned by the columns of B , which is an r -dimensional subspace of the output space $\mathbb{R}^k$. The row space of ΔW is spanned by the rows of A , which is an r -dimensional subspace of the input space $\mathbb{R}^d$. Thus, the adaptation updates are strictly constrained to these coordinates, preserving all other dimensions from degradation.

Visualizing the Calculations

We can visualize how the information bottleneck constrains the updates in the representation space during the forward and backward passes:

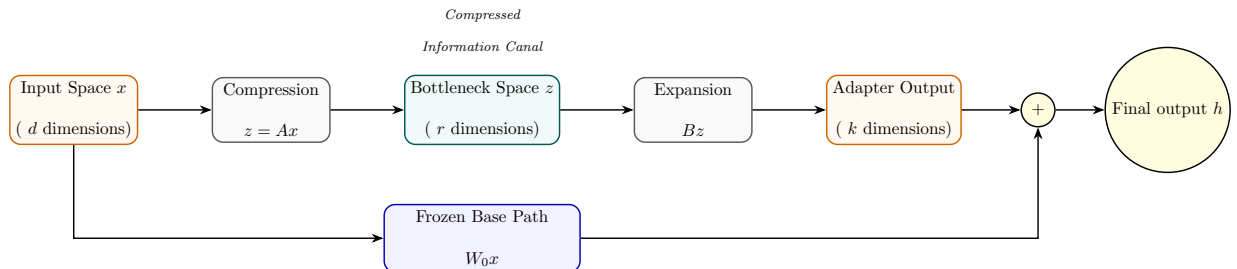


Figure 15: Visualization of the information bottleneck: high-dimensional input is projected into a low-dimensional canal where task-specific tuning forces are applied, then projected back.

- **Forward Pass Compression:** The activation vector $x \in \mathbb{R}^{d \times 1}$ is projected down through a narrow bottleneck of rank r . Since $r \ll \min(d, k)$, the projection filters out dimensions irrelevant to the current task, creating a compressed task representation $z \in \mathbb{R}^{r \times 1}$.

- **Backward Pass Gradient Flow:** Error gradients $\nabla_h \mathcal{L}$ flow backwards. Because W_0 is frozen and lacks a gradient buffer, no gradient update calculations occur on the base weight matrix. Instead, the gradients flow down the low-rank adapters B and A , computing updates specifically for the low-rank subspace and avoiding parameter updates across the whole network.

The Physical and Philosophical Foundations of LoRA

Philosophical Foundations: Occam’s Razor and the Manifold Hypothesis

The success of Low-Rank Adaptation (LoRA) is not merely an empirical coincidence, but is deeply rooted in two foundational philosophical principles of complexity. We analyze each principle through a dialectic framework of Arguments, Counter-Arguments, and Rebuttals:

1. The Manifold Hypothesis

- **Argument:** This hypothesis posits that high-dimensional data (such as the activations in a neural network) actually lies on a much lower-dimensional, highly structured manifold. Pretraining constructs this manifold by mapping the topological structure of natural language.

In the context of Large Language Models, the embedding space has an extrinsic dimension of $d = 4096$ (e.g., $d_{\text{model}} = 4096$). However, the set of all grammatically correct and semantically coherent sentences does not fill this space uniformly; instead, it is confined to a lower-dimensional manifold. When we adapt the model to a downstream task (e.g., teaching it Python syntax or chat style), we do not need to construct a new manifold. We only need to compute a small trajectory or translation vector along the existing surface. This translation vector corresponds to a tangent vector in the manifold’s tangent bundle. By confining our search to this tangent space, we avoid distorting the global topology of the language manifold, preventing catastrophic forgetting.

- **Counter-Argument:** If the downstream task represents an extreme domain shift (e.g., shifting from English literature to highly specialized biochemistry nomenclature or low-level assembly code), the target representations may lie completely outside the manifold mapped during pretraining. In such cases, the translation vector cannot lie along the existing manifold surface, and restricting the updates to a low-rank tangent space will lead to underfitting. The model would fail to acquire the new features because the required coordinates do not exist in the pretrained column space.
- **Rebuttal / Synthesis:** While severe domain shifts can strain low-rank constraints, empirical evidence shows that even highly specialized knowledge maps back to general-purpose linguistic structures (syntax, logic, context). Furthermore, if the target lies slightly off-manifold, a small rank r (e.g., 8 to 16) is still sufficient to project the input into the nearest point on the task manifold. If needed, the user can scale the rank r to increase the degrees of freedom, which mathematically expands the coordinate projection space, allowing the model to capture features that lie slightly outside the pretraining manifold while still regularizing the update to prevent arbitrary drift.

2. Occam’s Razor (Lex Parsimoniae)

- **Argument:** Plurality should not be posited without necessity. Full fine-tuning searches an extremely large, redundant parameter space ($\mathbb{R}^{k \times d}$), which contains millions of redundant degrees of freedom. This massive parameter space allows the model to find arbitrary, highly complex fitting functions that overfit the downstream training data and overwrite pretrained general knowledge.

By restricting the learning search to a compressed subspace ($\mathbb{R}^{r \times (d+k)}$), LoRA acts as a mathematical regularization agent. It forces the model to find the simplest, most parsimonious path of adaptation. This acts as a geometric constraint (projection regularization), restricting updates to only the principal directions of adaptation, thereby preserving the model’s core foundational capabilities.

- **Counter-Argument:** By restricting the parameters to a low-rank product BA , we might force the model to find an oversimplified adaptation path that fails to capture the subtle, non-linear correlations required for complex reasoning tasks. Furthermore, Occam’s Razor assumes that simpler models generalize better, but in deep learning, the phenomenon of “double descent” shows that highly overparameterized models can generalize better than simpler ones by finding smooth interpolating functions in high-dimensional spaces.
- **Rebuttal / Synthesis:** Double descent occurs when the entire network is overparameterized and trained on vast, diverse datasets where it can interpolate smoothly. However, in downstream fine-tuning, the dataset is small and highly focused. In this regime, overparameterization leads to catastrophic forgetting and representation collapse. Constraining the updates to a low-rank subspace restricts the model from modifying the general-purpose features, while allowing sufficient capacity to learn the task. Thus, LoRA combines the benefits of high-dimensional pretraining (smooth interpolation on general data) with low-dimensional adaptation (strict regularization on task-specific data).

Internal Mechanics and Representation Space Geometry

To understand why LoRA works under the hood, we must analyze the geometry of representation space and the intrinsic dimension of neural networks:

- **Intrinsic vs. Extrinsic Dimension:** A pretrained model has a very high extrinsic dimension (the number of weights $W_0 \in \mathbb{R}^{k \times d}$). However, empirical work by Aghajanyan et al. (2020) demonstrated that the parameter space of pretrained language models has a much lower *intrinsic dimension*. That is, the directions in weight space that significantly affect the loss are highly concentrated on a low-dimensional subspace. This is why we can optimize a tiny fraction of parameters and achieve the same or better task performance.
- **Subspace Alignment and Trajectory Geometry:** The activation vectors x represent coordinates on the manifold. The base weight matrix W_0 maps these coordinates to the next representation space. The update matrix $\Delta W = \frac{\alpha}{r}BA$ acts as a linear projection operator that projects the activations into a tiny subspace of dimension r via A , rotates/scales them in this bottleneck space, and then projects them back to the output space via B . This operation restricts the adaptation to a set of r directional vectors. Instead of shifting the entire coordinate frame, the adapter only redirects the activation trajectory along the principal directions relevant to the downstream task.

Geometrically, the column space of ΔW is spanned by the columns of B , which is an r -dimensional subspace of the output space $\mathbb{R}^k$. The row space of ΔW is spanned by the rows of A , which is an r -dimensional subspace of the input space $\mathbb{R}^d$. Thus, the adaptation updates are strictly constrained to these coordinates, preserving all other dimensions from degradation.

Visualizing the Calculations

We can visualize how the information bottleneck constrains the updates in the representation space during the forward and backward passes:

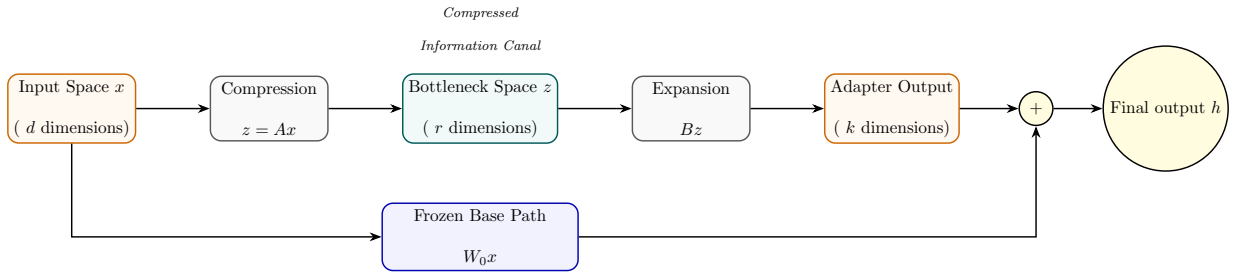


Figure 16: Visualization of the information bottleneck: high-dimensional input is projected into a low-dimensional canal where task-specific tuning forces are applied, then projected back.

- **Forward Pass Compression:** The activation vector $x \in \mathbb{R}^{d \times 1}$ is projected down through a narrow bottleneck of rank r . Since $r \ll \min(d, k)$, the projection filters out dimensions irrelevant to the current task, creating a compressed task representation $z \in \mathbb{R}^{r \times 1}$.
- **Backward Pass Gradient Flow:** Error gradients $\nabla_h \mathcal{L}$ flow backwards. Because W_0 is frozen and lacks a gradient buffer, no gradient update calculations occur on the base weight matrix. Instead, the gradients flow down the low-rank adapters B and A , computing updates specifically for the low-rank subspace and avoiding parameter updates across the whole network.

The Physical and Philosophical Foundations of LoRA

Philosophical Foundations: Occam’s Razor and the Manifold Hypothesis

The success of Low-Rank Adaptation (LoRA) is not merely an empirical coincidence, but is deeply rooted in two foundational philosophical principles of complexity. We analyze each principle through a dialectic framework of Arguments, Counter-Arguments, and Rebuttals:

1. The Manifold Hypothesis

- **Argument:** This hypothesis posits that high-dimensional data (such as the activations in a neural network) actually lies on a much lower-dimensional, highly structured manifold. Pretraining constructs this manifold by mapping the topological structure of natural language.

In the context of Large Language Models, the embedding space has an extrinsic dimension of $d = 4096$ (e.g., $d_{\text{model}} = 4096$). However, the set of all grammatically correct and semantically coherent sentences does not fill

this space uniformly; instead, it is confined to a lower-dimensional manifold. When we adapt the model to a downstream task (e.g., teaching it Python syntax or chat style), we do not need to construct a new manifold. We only need to compute a small trajectory or translation vector along the existing surface. This translation vector corresponds to a tangent vector in the manifold’s tangent bundle. By confining our search to this tangent space, we avoid distorting the global topology of the language manifold, preventing catastrophic forgetting.

- **Counter-Argument:** If the downstream task represents an extreme domain shift (e.g., shifting from English literature to highly specialized biochemistry nomenclature or low-level assembly code), the target representations may lie completely outside the manifold mapped during pretraining. In such cases, the translation vector cannot lie along the existing manifold surface, and restricting the updates to a low-rank tangent space will lead to underfitting. The model would fail to acquire the new features because the required coordinates do not exist in the pretrained column space.
- **Rebuttal / Synthesis:** While severe domain shifts can strain low-rank constraints, empirical evidence shows that even highly specialized knowledge maps back to general-purpose linguistic structures (syntax, logic, context). Furthermore, if the target lies slightly off-manifold, a small rank r (e.g., 8 to 16) is still sufficient to project the input into the nearest point on the task manifold. If needed, the user can scale the rank r to increase the degrees of freedom, which mathematically expands the coordinate projection space, allowing the model to capture features that lie slightly outside the pretraining manifold while still regularizing the update to prevent arbitrary drift.

2. Occam’s Razor (Lex Parsimoniae)

- **Argument:** Plurality should not be posited without necessity. Full fine-tuning searches an extremely large, redundant parameter space ($\mathbb{R}^{k \times d}$), which contains millions of redundant degrees of freedom. This massive parameter space allows the model to find arbitrary, highly complex fitting functions that overfit the downstream training data and overwrite pretrained general knowledge.

By restricting the learning search to a compressed subspace ($\mathbb{R}^{r \times (d+k)}$), LoRA acts as a mathematical regularization agent. It forces the model to find the simplest, most parsimonious path of adaptation. This acts as a geometric constraint (projection regularization), restricting updates to only the principal directions of adaptation, thereby preserving the model’s core foundational capabilities.

- **Counter-Argument:** By restricting the parameters to a low-rank product BA , we might force the model to find an oversimplified adaptation path that fails to capture the subtle, non-linear correlations required for complex reasoning tasks. Furthermore, Occam’s Razor assumes that simpler models generalize better, but in deep learning, the phenomenon of "double descent" shows that highly overparameterized models can generalize better than simpler ones by finding smooth interpolating functions in high-dimensional spaces.
- **Rebuttal / Synthesis:** Double descent occurs when the entire network is overparameterized and trained on vast, diverse datasets where it can interpolate smoothly. However, in downstream fine-tuning, the dataset is small and highly focused. In this regime, overparameterization leads to catastrophic forgetting and representation collapse. Constraining the updates to a low-rank subspace restricts the model from modifying the general-purpose features, while allowing sufficient capacity to learn the task. Thus, LoRA combines the benefits of high-dimensional pretraining (smooth interpolation on general data) with low-dimensional adaptation (strict regularization on task-specific data).

Internal Mechanics and Representation Space Geometry

To understand why LoRA works under the hood, we must analyze the geometry of representation space and the intrinsic dimension of neural networks:

- **Intrinsic vs. Extrinsic Dimension:** A pretrained model has a very high extrinsic dimension (the number of weights $W_0 \in \mathbb{R}^{k \times d}$). However, empirical work by Aghajanyan et al. (2020) demonstrated that the parameter space of pretrained language models has a much lower *intrinsic dimension*. That is, the directions in weight space that significantly affect the loss are highly concentrated on a low-dimensional subspace. This is why we can optimize a tiny fraction of parameters and achieve the same or better task performance.
- **Subspace Alignment and Trajectory Geometry:** The activation vectors x represent coordinates on the manifold. The base weight matrix W_0 maps these coordinates to the next representation space. The update matrix $\Delta W = \frac{\alpha}{r} BA$ acts as a linear projection operator that projects the activations into a tiny subspace of dimension r via A , rotates/scales them in this bottleneck space, and then projects them back to the output space via B . This operation restricts the adaptation to a set of r directional vectors. Instead of shifting the entire coordinate frame, the adapter only redirects the activation trajectory along the principal directions relevant to the downstream task.

Geometrically, the column space of ΔW is spanned by the columns of B , which is an r -dimensional subspace of the output space $\mathbb{R}^k$. The row space of ΔW is spanned by the rows of A , which is an r -dimensional subspace of the input space $\mathbb{R}^d$. Thus, the adaptation updates are strictly constrained to these coordinates, preserving all other dimensions from degradation.

Visualizing the Calculations

We can visualize how the information bottleneck constrains the updates in the representation space during the forward and backward passes:

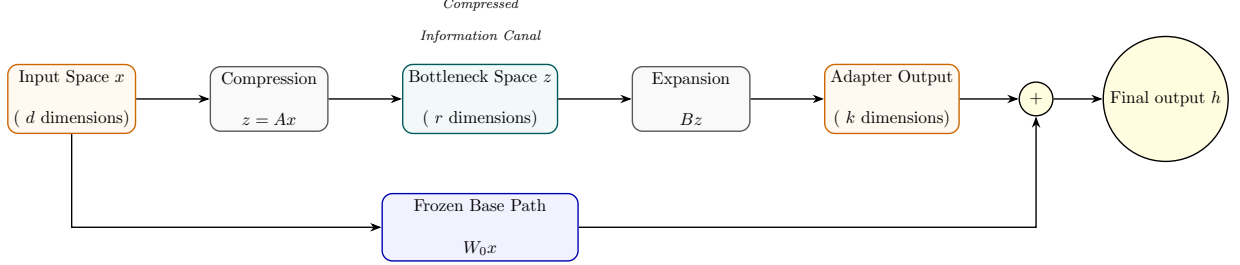


Figure 17: Visualization of the information bottleneck: high-dimensional input is projected into a low-dimensional canal where task-specific tuning forces are applied, then projected back.

- **Forward Pass Compression:** The activation vector $x \in \mathbb{R}^{d \times 1}$ is projected down through a narrow bottleneck of rank r . Since $r \ll \min(d, k)$, the projection filters out dimensions irrelevant to the current task, creating a compressed task representation $z \in \mathbb{R}^{r \times 1}$.
- **Backward Pass Gradient Flow:** Error gradients $\nabla_h \mathcal{L}$ flow backwards. Because W_0 is frozen and lacks a gradient buffer, no gradient update calculations occur on the base weight matrix. Instead, the gradients flow down the low-rank adapters B and A , computing updates specifically for the low-rank subspace and avoiding parameter updates across the whole network.

The Physical and Philosophical Foundations of LoRA

Philosophical Foundations: Occam’s Razor and the Manifold Hypothesis

The success of Low-Rank Adaptation (LoRA) is not merely an empirical coincidence, but is deeply rooted in two foundational philosophical principles of complexity. We analyze each principle through a dialectic framework of Arguments, Counter-Arguments, and Rebuttals:

1. The Manifold Hypothesis

- **Argument:** This hypothesis posits that high-dimensional data (such as the activations in a neural network) actually lies on a much lower-dimensional, highly structured manifold. Pretraining constructs this manifold by mapping the topological structure of natural language.

In the context of Large Language Models, the embedding space has an extrinsic dimension of $d = 4096$ (e.g., $d_{\text{model}} = 4096$). However, the set of all grammatically correct and semantically coherent sentences does not fill this space uniformly; instead, it is confined to a lower-dimensional manifold. When we adapt the model to a downstream task (e.g., teaching it Python syntax or chat style), we do not need to construct a new manifold. We only need to compute a small trajectory or translation vector along the existing surface. This translation vector corresponds to a tangent vector in the manifold’s tangent bundle. By confining our search to this tangent space, we avoid distorting the global topology of the language manifold, preventing catastrophic forgetting.

- **Counter-Argument:** If the downstream task represents an extreme domain shift (e.g., shifting from English literature to highly specialized biochemistry nomenclature or low-level assembly code), the target representations may lie completely outside the manifold mapped during pretraining. In such cases, the translation vector cannot lie along the existing manifold surface, and restricting the updates to a low-rank tangent space will lead to underfitting. The model would fail to acquire the new features because the required coordinates do not exist in the pretrained column space.
- **Rebuttal / Synthesis:** While severe domain shifts can strain low-rank constraints, empirical evidence shows that even highly specialized knowledge maps back to general-purpose linguistic structures (syntax, logic, context). Furthermore, if the target lies slightly off-manifold, a small rank r (e.g., 8 to 16) is still sufficient to project the input into the nearest point on the task manifold. If needed, the user can scale the rank r to increase the degrees of freedom, which mathematically expands the coordinate projection space, allowing the model to

capture features that lie slightly outside the pretraining manifold while still regularizing the update to prevent arbitrary drift.

2. Occam’s Razor (Lex Parsimoniae)

- **Argument:** Plurality should not be posited without necessity. Full fine-tuning searches an extremely large, redundant parameter space ($\mathbb{R}^{k \times d}$), which contains millions of redundant degrees of freedom. This massive parameter space allows the model to find arbitrary, highly complex fitting functions that overfit the downstream training data and overwrite pretrained general knowledge.

By restricting the learning search to a compressed subspace ($\mathbb{R}^{r \times (d+k)}$), LoRA acts as a mathematical regularization agent. It forces the model to find the simplest, most parsimonious path of adaptation. This acts as a geometric constraint (projection regularization), restricting updates to only the principal directions of adaptation, thereby preserving the model’s core foundational capabilities.

- **Counter-Argument:** By restricting the parameters to a low-rank product BA , we might force the model to find an oversimplified adaptation path that fails to capture the subtle, non-linear correlations required for complex reasoning tasks. Furthermore, Occam’s Razor assumes that simpler models generalize better, but in deep learning, the phenomenon of “double descent” shows that highly overparameterized models can generalize better than simpler ones by finding smooth interpolating functions in high-dimensional spaces.
- **Rebuttal / Synthesis:** Double descent occurs when the entire network is overparameterized and trained on vast, diverse datasets where it can interpolate smoothly. However, in downstream fine-tuning, the dataset is small and highly focused. In this regime, overparameterization leads to catastrophic forgetting and representation collapse. Constraining the updates to a low-rank subspace restricts the model from modifying the general-purpose features, while allowing sufficient capacity to learn the task. Thus, LoRA combines the benefits of high-dimensional pretraining (smooth interpolation on general data) with low-dimensional adaptation (strict regularization on task-specific data).

Internal Mechanics and Representation Space Geometry

To understand why LoRA works under the hood, we must analyze the geometry of representation space and the intrinsic dimension of neural networks:

- **Intrinsic vs. Extrinsic Dimension:** A pretrained model has a very high extrinsic dimension (the number of weights $W_0 \in \mathbb{R}^{k \times d}$). However, empirical work by Aghajanyan et al. (2020) demonstrated that the parameter space of pretrained language models has a much lower *intrinsic dimension*. That is, the directions in weight space that significantly affect the loss are highly concentrated on a low-dimensional subspace. This is why we can optimize a tiny fraction of parameters and achieve the same or better task performance.
- **Subspace Alignment and Trajectory Geometry:** The activation vectors x represent coordinates on the manifold. The base weight matrix W_0 maps these coordinates to the next representation space. The update matrix $\Delta W = \frac{\alpha}{r}BA$ acts as a linear projection operator that projects the activations into a tiny subspace of dimension r via A , rotates/scales them in this bottleneck space, and then projects them back to the output space via B . This operation restricts the adaptation to a set of r directional vectors. Instead of shifting the entire coordinate frame, the adapter only redirects the activation trajectory along the principal directions relevant to the downstream task.

Geometrically, the column space of ΔW is spanned by the columns of B , which is an r -dimensional subspace of the output space $\mathbb{R}^k$. The row space of ΔW is spanned by the rows of A , which is an r -dimensional subspace of the input space $\mathbb{R}^d$. Thus, the adaptation updates are strictly constrained to these coordinates, preserving all other dimensions from degradation.

Visualizing the Calculations

We can visualize how the information bottleneck constrains the updates in the representation space during the forward and backward passes:

- **Forward Pass Compression:** The activation vector $x \in \mathbb{R}^{d \times 1}$ is projected down through a narrow bottleneck of rank r . Since $r \ll \min(d, k)$, the projection filters out dimensions irrelevant to the current task, creating a compressed task representation $z \in \mathbb{R}^{r \times 1}$.
- **Backward Pass Gradient Flow:** Error gradients $\nabla_h \mathcal{L}$ flow backwards. Because W_0 is frozen and lacks a gradient buffer, no gradient update calculations occur on the base weight matrix. Instead, the gradients flow down the low-rank adapters B and A , computing updates specifically for the low-rank subspace and avoiding parameter updates across the whole network.

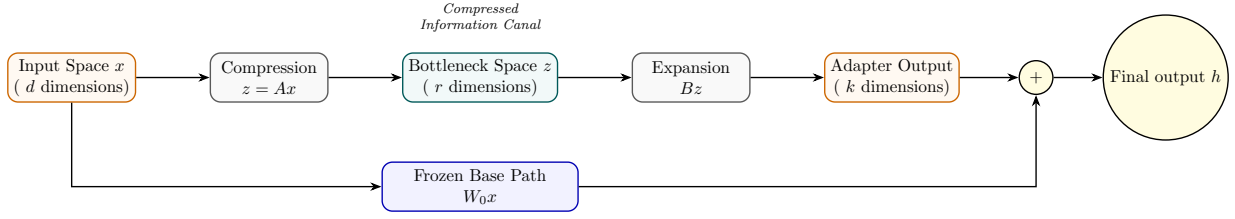


Figure 18: Visualization of the information bottleneck: high-dimensional input is projected into a low-dimensional canal where task-specific tuning forces are applied, then projected back.

LoRA CLRS-Style Algorithms

This section contains the CLRS-style mathematical algorithms for the forward pass, manual backward pass, parameter merging, and AdamW optimizer update step.

```

LORA-FORWARD( $x, W_0, A, B, \alpha, r$ )
   $h_{\text{base}} = W_0 x$ 
   $z = Ax$ 
   $z_{\text{out}} = Bz$ 
   $h = h_{\text{base}} + \frac{\alpha}{r} z_{\text{out}}$ 
  return  $h$ 

```

Figure 19: LORA-FORWARD algorithm: computes base output and scaled low-rank adapter output.

Analysis of Lora-Forward

- **What is achieved:** Computes the layer output h by parallelizing the computation of the frozen base weight path ($W_0 x$) and the trainable low-rank adapter path ($B(Ax)$), and scaling the adapter's contribution.
- **What internally changed:** Instantiates intermediate activation tensors $z = Ax \in \mathbb{R}^{r \times 1}$ and $z_{\text{out}} = Bz \in \mathbb{R}^{k \times 1}$ in memory. During training, these tensors are kept in GPU memory (VRAM) because their values are required to compute the gradients in the backward pass.
- **User-modifiable parameters & impact:**
 - **Rank r** (hyperparameter): Modifies the bottleneck dimension. A larger r (e.g., 32) captures more task-specific information but increases memory requirements and latency. A smaller r (e.g., 8) is highly memory-efficient but might underfit complex tasks.
 - **Scaling constant α** (hyperparameter): Adjusts the strength of the adapter's adjustments. Increasing α scales up the adapter's updates. Keep this constant when tuning r to stabilize learning.

```

LORA-BACKWARD( $x, A, B, \nabla_h \mathcal{L}, \alpha, r$ )
   $z = Ax$ 
   $\nabla_B \mathcal{L} = \frac{\alpha}{r} (\nabla_h \mathcal{L}) z^\top$ 
   $\nabla_z \mathcal{L} = \frac{\alpha}{r} B^\top \nabla_h \mathcal{L}$ 
   $\nabla_A \mathcal{L} = (\nabla_z \mathcal{L}) x^\top$ 
   $\nabla_{x, \text{adapters}} \mathcal{L} = A^\top \nabla_z \mathcal{L}$ 
  return  $\nabla_A \mathcal{L}, \nabla_B \mathcal{L}, \nabla_{x, \text{adapters}} \mathcal{L}$ 

```

Figure 20: LORA-BACKWARD algorithm: derives gradients for adapters and backpropagated input gradient.

Analysis of Lora-Backward

- **What is achieved:** Calculates the exact gradients $\nabla_A \mathcal{L}$ and $\nabla_B \mathcal{L}$ for the adapter weights, and computes the backpropagated input gradient $\nabla_{x, \text{adapters}} \mathcal{L}$ to pass back to previous layers.
- **What internally changed:** Populates the gradient buffers **A.grad** and **B.grad** in VRAM. The frozen weights W_0 do not have gradient buffers, saving $d \times k \times 2$ or 4 bytes of memory.
- **User-modifiable parameters & impact:**
 - **Rank r and Scaling α :** Directly scale the gradients via the factor $\frac{\alpha}{r}$. Modifying them scales the gradient step size, affecting convergence stability.

```

LORA-MERGE( $W_0, A, B, \alpha, r, mode$ )
 $\Delta W = \frac{\alpha}{r} BA$ 
if  $mode == \text{"merge"}$ 
     $W = W_0 + \Delta W$ 
else if  $mode == \text{"demerge"}$ 
     $W = W_0 - \Delta W$ 
return  $W$ 

```

Figure 21: LORA-MERGE algorithm: handles parameter folding and unfolding for zero-latency inference.

Analysis of Lora-Merge

- **What is achieved:** Merges the low-rank adapters directly into the base weights ($W = W_0 \pm \Delta W$) for zero-latency inference, or subtracts them to restore the base weights.
- **What internally changed:** In-place updates are applied to the base weight tensor W_0 in memory. Once merged, the adapter weights A and B can be discarded, and the model behaves as a single unified feedforward network, eliminating the need to compute separate low-rank paths during inference.
- **User-modifiable parameters & impact:**
 - **Mode:** User can choose "merge" to prepare the model for high-throughput deployment, or "demerge" to unpack the adapters and swap in a different set of adapter weights.

```

LORA-ADAMW-UPDATE( $\theta, g, m, v, \eta, \lambda, \beta_1, \beta_2, \epsilon, t$ )
for each parameter matrix  $P \in \theta$  do
     $P = P - \eta \lambda P$ 
     $m_P = \beta_1 m_P + (1 - \beta_1) g_P$ 
     $v_P = \beta_2 v_P + (1 - \beta_2) g_P^2$ 
     $\hat{m}_P = m_P / (1 - \beta_1^t)$ 
     $\hat{v}_P = v_P / (1 - \beta_2^t)$ 
     $P = P - \eta \frac{\hat{m}_P}{\sqrt{\hat{v}_P} + \epsilon}$ 
return  $\theta, m, v$ 

```

Figure 22: LORA-ADAMW-UPDATE algorithm: updates parameters with decoupled weight decay and momentum/-variance tracking.

Analysis of Lora-AdamW-Update

- **What is achieved:** Applies decoupled weight decay to the adapter parameters and updates their values using momentum and variance of gradients.
- **What internally changed:** Trainable parameters A and B are updated. Additionally, the first moment vectors m_A, m_B and second moment vectors v_A, v_B are updated in GPU memory. Because W_0 is frozen, no optimizer states are created or updated for W_0 , resulting in huge VRAM savings.
- **User-modifiable parameters & impact:**
 - **Learning Rate η :** Controls the step size. High values speed up training but risk divergence; low values ensure stability but slow down convergence.
 - **Weight Decay λ :** Regularizes the weights to prevent overfitting.
 - **Momentum coefficients β_1, β_2 :** Control the memory of past gradients.

Quantized LoRA (QLoRA)

Mathematical Foundation

QLoRA quantizes the base weight W_0 to 4-bit NormalFloat (NF4) and uses Double Quantization (DQ) to minimize memory requirements. NF4 divides the standard normal distribution $\mathcal{N}(0, 1)$ into $2^k = 16$ equal-area intervals. The quantization levels q_i are calculated using the standard normal quantile function Q_X :

$$q_i = \frac{1}{2} \left(Q_X \left(\frac{i}{2^k} \right) + Q_X \left(\frac{i+1}{2^k} \right) \right) \quad (42)$$

The base weights are normalized to $[-1, 1]$ using block-wise scaling. Let the block size be $B = 64$. For a block W_b , the scale is $c_b = \max(|W_b|)$. The quantized values are:

$$q_t = \arg \min_{i \in \{0, \dots, 15\}} \left| \frac{W_{b,t}}{c_b} - q_i \right| \quad (43)$$

Double Quantization (DQ) quantizes the scaling factors c_b themselves to 8-bit floats (c_b^{FP8}) with a second scale block size of 256, reducing scale memory overhead from $32/64 = 0.5$ bits/weight to $8/64 + 32/(64 \times 256) \approx 0.127$ bits/weight. During backpropagation, the NF4 weights are dequantized to FP16/BF16 to compute outputs, and gradients are computed only for the low-rank parameters:

$$h = \text{dequant}(c_b, W^{\text{NF4}})x + \frac{\alpha}{r} B A x \quad (44)$$

Architecture Flow Diagram

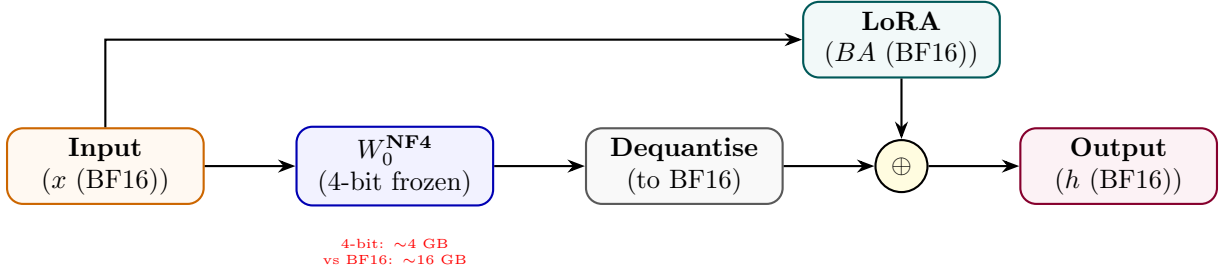


Figure 23: QLoRA: 4-bit frozen base dequantised at runtime; LoRA adapters trained in BF16.

State Transition Table

Property	Before	After
W_0 precision	BF16 / FP16	NF4 (4-bit)
GPU memory for 70B model	~140 GB	~35 GB
LoRA adapter precision	N/A	BF16
Training fidelity	Full	Near-full (straight-through)
Inference latency	Baseline	+dequant overhead

Table 12: State properties before and after QLoRA.

Adaptive LoRA (AdaLoRA)

Mathematical Foundation

AdaLoRA parameterizes incremental weight updates using a singular value decomposition (SVD) representation to dynamically adjust the adapter rank during training:

$$\Delta W = P\Lambda Q \quad (45)$$

where $P \in \mathbb{R}^{d \times r}$ and $Q \in \mathbb{R}^{r \times k}$ are left and right singular vectors, and $\Lambda = \text{diag}(\lambda_1, \dots, \lambda_r) \in \mathbb{R}^{r \times r}$ contains singular values. To prevent SVD divergence and maintain parameter stability, we enforce orthogonality constraints:

$$\mathcal{L}_{\text{orth}}(P, Q) = \gamma (\|P^\top P - I\|_F^2 + \|QQ^\top - I\|_F^2) \quad (46)$$

We estimate the training importance of each singular triplet $(P_{*,i}, \lambda_i, Q_{i,*})$ using gradient-weight product magnitude:

$$S_i = s(\lambda_i) + \frac{1}{d} \sum_{j=1}^d s(P_{j,i}) + \frac{1}{k} \sum_{j=1}^k s(Q_{i,j}) \quad (47)$$

where $s(\theta) = |\theta \cdot \nabla_\theta \mathcal{L}|$. At designated intervals, triplets with the lowest importance scores S_i are pruned by zeroing out their singular values λ_i , dynamically reducing the adapter rank budget.

Architecture Flow Diagram

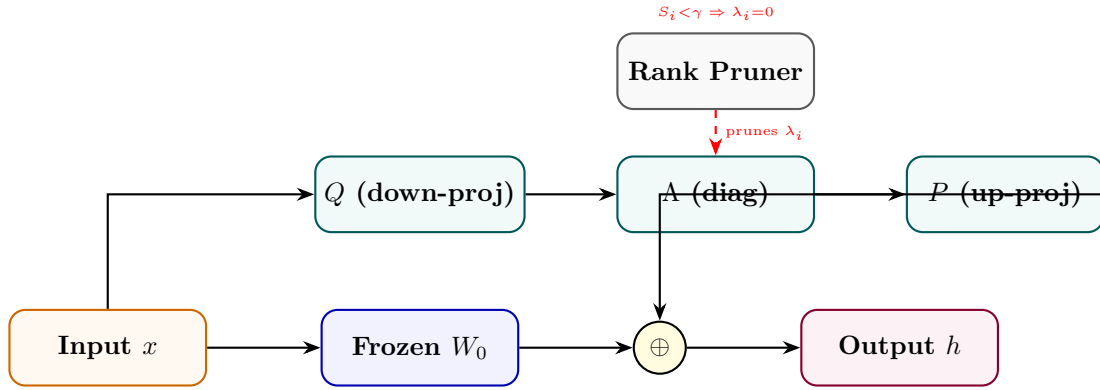


Figure 24: AdaLoRA: SVD-decomposed update with importance-based rank pruning per layer.

State Transition Table

Property	Before	After
Rank per layer	Fixed r	Adaptive (varies by importance)
Update structure	BA product	PAQ SVD triplet
Parameter budget	Fixed	Redistributed across layers
Low-importance ranks	Included	Pruned ($\lambda_i = 0$)
Training stability	Standard	Constrained by SVD orthogonality

Table 13: State properties before and after AdaLoRA.

Weight-Decomposed LoRA (DoRA)

Mathematical Foundation

DoRA decomposes the weight matrix $W \in \mathbb{R}^{k \times d}$ into its magnitude vector $m \in \mathbb{R}^k$ and direction matrix $V \in \mathbb{R}^{k \times d}$:

$$W = m \cdot \frac{V}{\|V\|_r} = m \cdot \frac{W_0 + \Delta W}{\|W_0 + \Delta W\|_r} \quad (48)$$

where $\|\cdot\|_r$ is the L2 norm computed along each row (corresponding to output channels). In parameter-efficient training, we freeze the direction matrix to the base weight $V = W_0$ and introduce low-rank directional updates using adapters $\Delta V = \frac{\alpha}{r}BA$ (where $A \in \mathbb{R}^{r \times d}$ and $B \in \mathbb{R}^{k \times r}$):

$$W = m \cdot \frac{W_0 + \frac{\alpha}{r}BA}{\|W_0 + \frac{\alpha}{r}BA\|_r} \quad (49)$$

Only the magnitude vector m and low-rank directional matrices B and A are trainable. The gradient flow for the directional components is:

$$\nabla_V \mathcal{L} = \frac{m}{\|V\|_c} \left(\nabla_W \mathcal{L} - \frac{V}{\|V\|_c^2} \odot (V^\top \nabla_W \mathcal{L}) \right) \quad (50)$$

This mathematically separates the updates to magnitude and direction, mimicking full parameter fine-tuning behavior while retaining low parameter overhead.

Architecture Flow Diagram

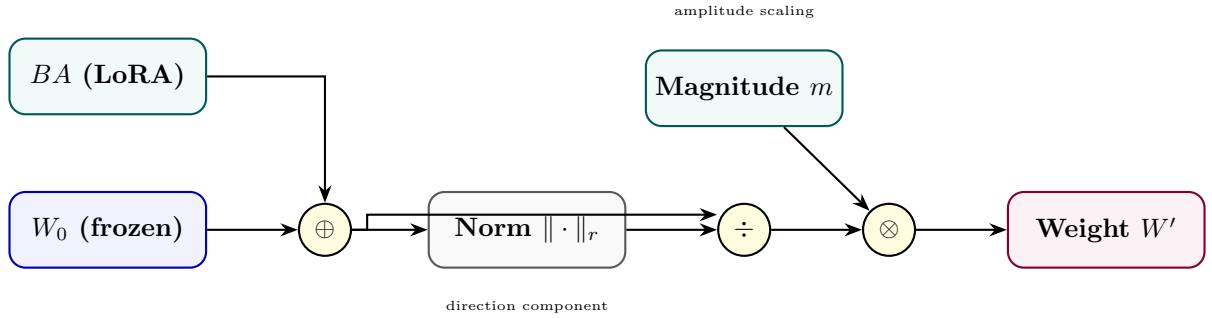


Figure 25: DoRA: weight decomposed into direction (LoRA-adapted) and magnitude (trainable scalar).

State Transition Table

Property	Before (LoRA)	After (DoRA)
Magnitude component	Implicit in W_0	Explicit trainable m
Direction component	$W_0 + BA$ unnorm	Column-normalised
Full FT similarity	Lower	Higher
Trainable params	$r(d + k)$	$r(d + k) + k$
Catastrophic forgetting	Low	Lower

Table 14: State properties comparing LoRA and DoRA.

IA³ (Infused Adapter by Inhibiting and Amplifying Inner Activations)

Mathematical Foundation

IA³ introduces element-wise scaling vectors to directly modify inner activations in attention keys, values, and intermediate feed-forward layers. Let the scaling vectors be $l_K \in \mathbb{R}^d$, $l_V \in \mathbb{R}^d$, and $l_{FFN} \in \mathbb{R}^{d_{FFN}}$. The attention layer activations are computed as:

$$\tilde{K} = K \odot l_K = K \cdot \text{diag}(l_K) \quad (51)$$

$$\tilde{V} = V \odot l_V = V \cdot \text{diag}(l_V) \quad (52)$$

$$\text{Attention}(Q, \tilde{K}, \tilde{V}) = \text{softmax}\left(\frac{Q\tilde{K}^\top}{\sqrt{d_k}}\right) \tilde{V} \quad (53)$$

In the feed-forward network (FFN), let W_1 and W_2 be the weight matrices. The intermediate activations are scaled before the output projection:

$$\text{FFN}(x) = \sigma(xW_1 \odot l_{FFN})W_2 = \sigma(xW_1 \cdot \text{diag}(l_{FFN}))W_2 \quad (54)$$

The base model weights W_Q, W_K, W_V, W_1, W_2 are frozen. This diagonal matrix transformation scales the columns of the projection matrices with minimal parameter updates.

Architecture Flow Diagram

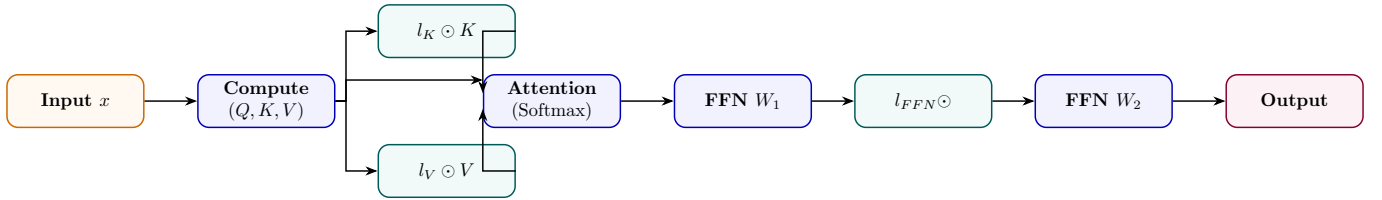


Figure 26: IA³: three tiny scaling vectors rescale keys, values, and FFN intermediate activations.

State Transition Table

Property	Before	After
Trainable params	0 (frozen)	$3dL$ scalars
Init value of l vectors	N/A	1 (identity behaviour)
K, V, FFN activations	Unchanged	Element-wise scaled
Param overhead vs LoRA	N/A	$\sim 10\times$ fewer params
Task adaptation strength	None	Moderate

Table 15: State properties before and after IA³ injection.

Adapter Tuning

Mathematical Foundation

Adapter tuning inserts small bottleneck layers into the transformer block, typically after attention and feed-forward modules. Let $h \in \mathbb{R}^d$ be the input hidden state. In the Pfeiffer architecture, a single adapter is inserted per block. In the Houlsby architecture, two adapters are inserted per block. The mathematical transformation for an adapter block is:

$$h' = h + \sigma(hW_{down})W_{up} \quad (55)$$

where $W_{down} \in \mathbb{R}^{d \times r}$ projects to a bottleneck dimension $r \ll d$, σ is an activation function (typically GELU), and $W_{up} \in \mathbb{R}^{r \times d}$ projects back to the original dimension. To ensure the adapter behaves as an identity mapping at the start of training, W_{up} is initialized to zero:

$$h'|_{t=0} = h + \sigma(hW_{down}) \cdot 0 = h \quad (56)$$

Architecture Flow Diagram

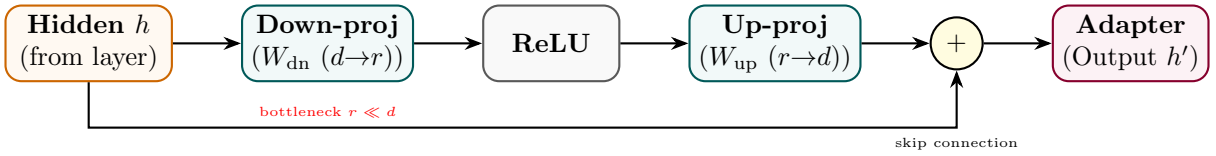


Figure 27: Adapter module: down-project to rank r , apply nonlinearity, up-project back, plus skip connection.

State Transition Table

Property	Before	After
Sub-layer output path	Direct pass-through	+adapter bottleneck
Transformer weights	Trainable	Frozen
New params per layer	0	$2dr + 2d$
Adapter init	N/A	Near-identity ($W_{dn} \sim \mathcal{N}$)
Inference latency	Baseline	+two extra matmuls per layer

Table 16: State properties before and after adapter insertion.

Prefix Tuning

Mathematical Foundation

Prefix Tuning prepends continuous task-specific virtual prefix vectors $P_K, P_V \in \mathbb{R}^{l \times d}$ directly to the Key and Value matrices at every transformer layer. Let the original Key and Value matrices for sequence length T be $K, V \in \mathbb{R}^{T \times d}$. The modified Key and Value matrices become:

$$\tilde{K} = [P_K; K] \in \mathbb{R}^{(l+T) \times d} \quad (57)$$

$$\tilde{V} = [P_V; V] \in \mathbb{R}^{(l+T) \times d} \quad (58)$$

To stabilize optimization, during training the prefix is reparameterized using a Multilayer Perceptron (MLP) over a small source embedding E_k :

$$P_K = \text{MLP}_K(E_k), \quad P_V = \text{MLP}_V(E_k) \quad (59)$$

where $E_k \in \mathbb{R}^{l \times d_{\text{mid}}}$. The MLP is discarded at inference, and only the generated static prefix matrices P_K, P_V are saved and used. The attention output is computed as:

$$\text{Attention}(Q, \tilde{K}, \tilde{V}) = \text{softmax}\left(\frac{Q[P_K; K]^\top}{\sqrt{d_k}}\right) [P_V; V] \quad (60)$$

Architecture Flow Diagram

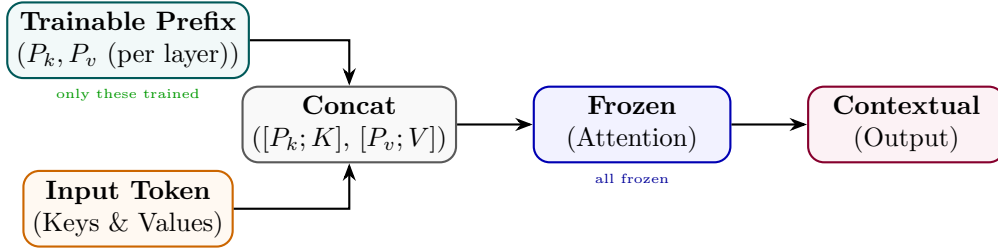


Figure 28: Prefix Tuning: trainable prefix vectors extend the key/value context at every attention layer.

State Transition Table

Property	Before	After
Attention context length	T tokens	$T + l$ tokens (prefix prepended)
Transformer params	Trainable	Fully frozen
Trainable params	0	$2 \times l \times d_h \times L$
Prefix init	N/A	Via MLP reparameterisation
Task switch at inference	Reload model	Swap prefix only

Table 17: State properties before and after prefix tuning.

Prompt Tuning

Mathematical Foundation

Prompt tuning prepends p learnable embedding vectors $P \in \mathbb{R}^{p \times d_{emb}}$ to the input word representations:

$$\tilde{X} = [P; X_e] \in \mathbb{R}^{(p+T) \times d_{emb}} \quad (61)$$

where $X_e \in \mathbb{R}^{T \times d_{emb}}$ is the original sequence word embedding matrix. The entire transformer backbone parameters θ_{base} remain frozen, and gradients are backpropagated only to the prompt parameters P :

$$\nabla_P \mathcal{L} = \frac{\partial \mathcal{L}}{\partial \tilde{X}_{1..p}} \quad (62)$$

During backpropagation, the gradients are passed through all layers of the frozen transformer and update only the prompt embeddings.

Architecture Flow Diagram

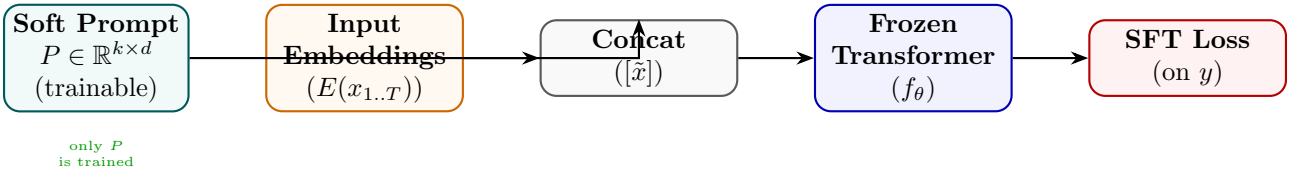


Figure 29: Prompt Tuning: soft tokens prepended at the embedding layer; the full transformer is frozen.

State Transition Table

Property	Before	After
Trainable params	Full model	$k \times d_{emb}$ only
Prompt format	Discrete hand-crafted	Continuous soft vectors
Transformer weights	All trainable	All frozen
Task switching	Reload fine-tuned model	Swap P only
Sensitivity to init	N/A	Strong (init from class labels helps)

Table 18: State properties before and after prompt tuning.

P-Tuning v2

Mathematical Foundation

P-Tuning v2 extends prompt tuning by prepending learnable virtual tokens $P^{(l)} \in \mathbb{R}^{p \times 2d_{\text{attn}}}$ directly to the attention key and value states at every layer $l \in [1, L]$ of the network. Unlike prefix tuning, P-Tuning v2 removes the MLP reparameterization network, directly optimizing the prefix parameters $P^{(l)}$:

$$\tilde{K}^{(l)} = [P_K^{(l)}; K^{(l)}] \quad (63)$$

$$\tilde{V}^{(l)} = [P_V^{(l)}; V^{(l)}] \quad (64)$$

This deep prefix injection resolves the capacity limitation of shallow prompt tuning, allowing small models ($< 10\text{B}$ parameters) to achieve performance comparable to full fine-tuning.

Architecture Flow Diagram

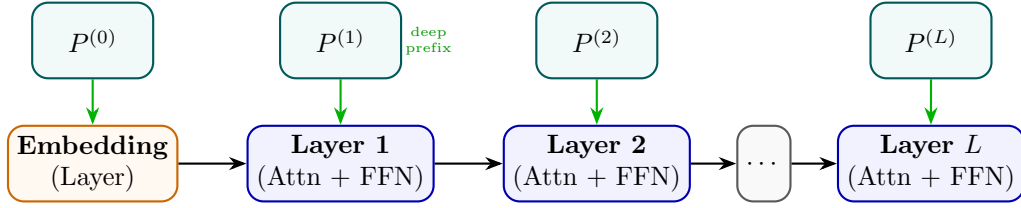


Figure 30: P-Tuning v2: independent trainable prefix tokens injected at every transformer layer.

State Transition Table

Property	Before (Prompt Tuning)	After (P-Tuning v2)
Prefix layers	Embedding only	All L layers
Trainable params	$k \times d$	$L \times l \times d$
NLU performance	Degrades at small scale	Competitive at all scales
Conditioning depth	Shallow (first layer)	Deep (every layer)
Task-specific storage	Single P matrix	L prefix matrices

Table 19: State properties comparing Prompt Tuning and P-Tuning v2.